Frozen-Weight Policy Transfer with Adaptable Mathematical Interfaces: A Benchmark Study on Differential-Drive Navigation

Given Name Surname, Given Name Surname
dept. name of organization (of Affiliation)
name of organization (of Affiliation)
City, Country
email address or ORCID

ABSTRACT— How far can a *frozen* learned controller be carried into new worlds when only lightweight mathematical interfaces are allowed to adapt? We answer on a differential-drive navigation benchmark in Isaac Lab / Isaac Sim with a ROS 2 deployment rung. Guarantees and competence come from different places, and neither arm alone does the job. A deploy-time safety shield, a discrete velocity-barrier filter that forward-simulates a 51-point hull over a 0.8 s horizon plus a braking tail and certifies every command against an obstacle model, makes contact-free operation structural: it holds by construction against that model. The shield is *arm-agnostic*. Wrapped around our in-house A*+DWA planner it scores 0.979 goal success with *zero* collisions in 236 campaign episodes, where the same planner unwrapped collides in 53.4 %; on a held-out maze drawn by a user and unseen in training it is 10/10 successful with 0/10 collisions (0/246 cumulative), where its unprotected twin collides in 9/10. The learned policy is the *only* arm that crosses dynamics and embodiment with frozen weights: a scalar disturbance observer restores degraded-actuator success from 0.028 to 0.363 (within 3.2 % of an oracle) with no weight change, and the same checkpoint drives a second embodiment (pioneer3dx) through a plain velocity topic. Its competence is family-structured, 0.94 on open map families and 0.00 on narrow corridors, and on the held-out maze it reaches 0/10. An adaptation-budget curve prices the frozen transfer at ≈238 k target-world demonstrations. We also state what the shield is *not*. Its guarantee is relative to the obstacle model (exact map geometry in batch simulation, persisted LiDAR evidence in ROS 2 deploy, so the same certification core is fed by two different perception layers) and to a hull box that under-covers the wheel envelope by ≈2 cm. That gap is the residual behind the campaign's single 5.6 N scrape. The contribution is a measured case study of *what a frozen learned core plus adaptable mathematical interfaces can and cannot do*. It is not a new planner, and it makes no claim that transfer learning is unnecessary. The seven-version evolution of the shield, from a five-map proof-of-concept to a 708-episode campaign, is reported as part of the result.

Keywords—Transfer learning, sim-to-real, frozen-weight deployment, control barrier functions, disturbance observers, Koopman operators, differential-drive navigation, safety filters.

I. INTRODUCTION

Deep reinforcement learning (DRL) produces capable robot controllers inside physics simulators. Deploying them outside the training world degrades their performance: the reality gap. The dominant remedies, domain randomization and target-domain fine-tuning, spend data and compute at every new deployment. Transfer learning in the classical sense (adapting weights with target data) already works for many of these problems, and we do not dispute it. The question this paper asks is narrower and, we argue, complementary: *how much of the cross-world burden can be carried without touching the weights at all?* If a policy trained once can be redeployed by adapting only small, interpretable, certifiable interfaces, then each new world costs interface work instead of a training campaign, and the safety-critical part of the stack becomes mathematical.

The operating philosophy is a *division of labour* between two kinds of component. A learned policy is good at the part that is hard to write down (threading a gap, recovering from a blip, steering competently in a world it has seen), but it cannot certify that it will never touch a wall, and its competence does not automatically survive a new geometry, a degraded actuator or a different robot. A mathematical interface is the complement. It is weak on its own, since a barrier filter reaches no goal, but it can be made to carry a *guarantee*, and it can be re-derived per deployment without retraining. The hypothesis this paper tests is that the two compose: *the learned policy supplies competence; the mathematical interface supplies the guarantees and task grounding that do not transfer by themselves; the weights never move.* This is the working thesis of the Neuro-Mathematical Transfer Engine (NMTE) program, first formulated as a mathematical architecture (a Koopman operator core, Gromov–Wasserstein cross-embodiment alignment, disturbance-observer-based control barrier functions) and instantiated here, deliberately modestly, on a 2-wheel robot with LiDAR, odometry and planar maps. The narrow scope is deliberate. It lets us measure the transfer-first paradigm end to end, including its failures, on hardware-class compute.

The measured result is this division of labour, and it does not come out as a clean win for either side. The guarantee arm is *arm-agnostic*: the same shield, with no per-controller tuning, makes a classical A*+DWA planner contact-free at scale (0/236 campaign collisions, 0/10 on a held-out user-drawn maze) where the planner alone collides in over half its episodes. The learned policy is the *only* arm that also crosses degraded dynamics and a second embodiment with frozen weights. On pure unseen geometry its competence runs out (0.00 on narrow corridors; 0/10 on the held-out maze), and there the shield-wrapped classical planner wins. We report that finding as it stands. A frozen learned core is not a drop-in replacement for a planner on arbitrary geometry; what it buys is a *portable* competence that, wrapped in a certifying interface, degrades safely instead of catastrophically.

The paper makes five concrete contributions, each backed by kept artifacts (traces, logs, scripts):

1. A frozen-weight transfer benchmark: one checkpoint, trained solely in a source world of scattered-wall rooms, evaluated across four shift axes (geometry, task structure, dynamics, embodiment) with zero gradient steps in any target world (Sec. VI).
2. A measured transfer gradient and boundary: a difficulty ladder with identical spawn, goal and path length shows that transfer degrades with *behavioural* distance from the source distribution, not with visual complexity; seeded repeats and a waypoint-blinding ablation separate robust transfer from lucky seeds and locate which interface carries it (Sec. VI-B).
3. A quantified adaptation budget: a retrain-from-scratch curve on the target world crosses the frozen engine's zero-shot performance at ≈238 k expert demonstrations, which puts a number on what transfer-first saves, with its caveats disclosed next to it (Sec. VI-B).
4. A deploy-time safety shield with a structural, arm-agnostic no-contact property: a three-tier velocity-barrier filter between controller and chassis makes “move, stop, do whatever you like, but never collide” true by construction against its obstacle model. The same certification core runs against two perception layers, an exact map-box distance field for high-throughput batch simulation and a persisted LiDAR grid for hardware-realistic ROS 2 deployment, which is direct evidence that the guarantee is decoupled from sensing. An exact-geometry audit verifies zero hull-penetration frames across a five-map presentation set; a three-arm campaign on 56 procedural maps (708 episodes) shows the guarantee is arm-agnostic (zero collisions for a shield-wrapped classical planner that collides in 53.4 % of episodes unwrapped, and gains success besides, 0.941 → 0.979); and the shield's own seven-version iteration history, including one livelock, one absorbing freeze, a hull-model gap and their fixes, is kept as evidence of what it takes to scale a safety filter from proof-of-concept to campaign (Sec. IV-D, VI-E, VI-G, VII-B).
5. A full accounting of the negatives. Failed training runs, a deadlocked shield version, a no-op safety-filter variant, an unhelpful Koopman lift, an unreachable map class and a marginal certified-box scrape traced to geometry the box does not cover are all reported alongside the positives, because the boundary of a transfer method is part of its result (Sec. VII-B).

The remainder of the paper is organised as follows. Sec. II positions the work against transfer learning, domain randomization and safety-filtered learning, and states precisely how our discrete filter differs from the standard CBF quadratic program. Sec. III describes how the framework idea and the shield itself evolved from the first draft to the benchmark. Sec. IV gives the system and methods, with the shield, observer, barrier and lift formalised as they are actually implemented. Sec. V defines the experimental setup and the audit methodology. Sec. VI reports results axis by axis. Sec. VII discusses what is proven, what is not, and how the framework serves the broader “learning can be transferred” goal. Sec. VIII concludes.

II. RELATED WORK AND POSITIONING

A. Transfer learning and domain randomization

Transfer learning (TL), the reuse of source-world knowledge with target-world adaptation, is an established and effective route across the reality gap: recent surveys treat it as the most promising direction for cross-embodiment robotics [1], and sim-to-real surveys cover the same ground for manipulation and locomotion [16]. Domain randomization (DR) trades target data for widened source distributions; it helps, but it leaves a residual distributional shift at deployment [2]. Cross-embodiment skill transfer through latent-space alignment is under active development [3], [18], [19]. Our work does not compete with any of these. It fixes the weights entirely and asks what interfaces alone can carry. Where TL spends target *data*, we spend target *structure* (a map file, an observer, a barrier); the adaptation-budget curve of Sec. VI-B is the exchange rate between the two paradigms on one axis.

B. Safety-filtered learning, and how our filter differs

Control barrier functions (CBFs) provide forward-invariance certificates for safe sets [9]. The standard way to combine a CBF with a learned policy is a minimally invasive quadratic program (QP): solve, at each step, $\min \|u - u_{nom}\|^2$ subject to a barrier constraint $\dot{h} \geq -\alpha(h)$ that requires an analytic relative-degree-one barrier h and its Lie derivative along the dynamics [4], [9]. Minkowski-type barriers extend this to polytopic footprints and obstacle sets [5]. At the planning level, the dynamic window approach (DWA) instead filters velocities by forward-simulated stoppability over a discrete command grid [8].

Our deploy-time shield belongs to the *DWA family, not the QP family*, and the difference is substantive. There is no analytic h, no relative-degree computation and no QP solver in the loop. Tier 1 *enumerates* a discrete command grid (8 linear × 7 angular velocities), forward-simulates the full 51-point hull along each constant-curvature arc plus a braking tail, keeps the candidates whose simulated hull stays outside the obstacle model, and among those picks the one closest to the policy's command. That is the *objective* of a CBF-QP, solved by enumeration over a grid instead of by a continuous solver. This buys three things the QP form does not give us for free. It certifies a *non-point hull with corners*: a disk or relative-degree-one barrier protects the sensor origin, not a 0.44 × 0.28 m box, and that is exactly the failure that killed our first shield version (Sec. IV-D). It needs no differentiable dynamics model, so the same code runs against an exact map field and against a noisy persisted-LiDAR grid. And it has no solver dependency, which matters for the edge-compute regime we target. The cost is conservatism and a discrete command resolution, which we quantify (Sec. VI-G). A separate minimal cone barrier (CBF-lite), used only for training-world concept checks, is likewise a closed-form speed clamp and not a QP (Sec. IV-C, VI-D).

C. Operator-theoretic dynamics models

Koopman operator theory lifts nonlinear dynamics into linearly evolving observables, enabling cheap linear prediction and control [6], [7], [14]. We use a frozen extended-dynamic-mode-decomposition (EDMD) lift as an *offline* concept check on the dynamics axis:

which channels of a frozen source-world model survive a dynamics shift, and what a small refit recovers (Sec. VI-C). We state the scope explicitly. This is a pipeline proof-of-existence, not a deployed controller; the lift is never closed around the plant in this paper. At this scale the quadratic lift adds little over an affine model. What survives is the *freeze-structure / adapt-interface* pattern; the lifting mechanism does not.

D. Positioning against the program's stated gaps

The first draft of this program identified five gaps: zero-shot transfer in contact-rich settings, task-invariant features under degraded terrain, parameter-efficient cross-embodiment alignment, certifiable safety during adaptation, and edge-compute constraints [1], [4], [16], [17]. This paper addresses the safety-certification gap with a deployed, audited, arm-agnostic mechanism; the geometry/task gaps with measured zero-shot transfer and ablations; the dynamics gap with a disturbance observer (deployed) and an operator-theoretic check (offline); and the edge gap with a measured shield cost of 3.6 ms per intervention tick on CPU against a 50 ms budget at 20 Hz. The cross-embodiment *alignment* gap (Gromov–Wasserstein optimal transport) is the one arm we do not exercise. It is moved to Future Work (Sec. VIII) and is not claimed here.

III. FROM FRAMEWORK DRAFT TO BENCHMARK: HOW THE IDEA EVOLVED

The first draft proposed NMTE as a mathematical architecture: a Koopman core for task-invariant dynamics, Gromov–Wasserstein (GW) optimal transport for cross-embodiment latent alignment, and disturbance-observer-based CBFs for certifiable safety, wrapped around a neural feature extractor. Building a first instantiation changed the framing in three ways, and we report the change.

First, the invariant is the division of labour, not the mechanisms. The named components (Koopman core, DOB, CBF, GW alignment) are concepts. In the benchmark each was instantiated in the simplest form that could be validated or falsified: a scalar plant-gain observer in place of a full disturbance observer, a discrete velocity-barrier filter in place of a CBF-QP, a 12-term quadratic EDMD lift used offline in place of a deployed Koopman controller, and GW not at all. What survived every substitution is the principle: *the learned policy provides competence; the mathematical interface provides guarantees and task grounding; the weights never move.* Fig. 1 states the principle as the system architecture: exactly one block is frozen, and every other block is an interface re-derived per deployment.

Second, competence itself needed an interface to exist. Pure PPO never bootstrapped blocked-goal reaching in the source world (Sec. VI-A); the policy became competent only after a geodesic-waypoint observation channel was introduced and a corrupted version of that channel was fixed. Transfer-first therefore starts earlier than deployment: the *training* interface (mathematical guidance in observation space) is what makes the frozen core worth transferring. The later waypoint-blinding ablation closes the loop, since removing that channel at deployment collapses transfer (Sec. VI-B).

Third, the guarantee arm had to be earned at deployment, and earning it took seven shield versions. The first presentation runs (iteration 1) deployed the competence arm alone and reached every goal while touching walls in almost every run; auditing that flaw (Sec. VI-E) quantified exactly the gap the framework draft had predicted, namely that learned policies cannot certify constraint satisfaction. Iteration 2 added a shield and re-ran the same maps with the same weights, and contact went to zero. A five-map proof-of-concept and a 708-episode campaign are not the same object, however, and the distance between them is where the real engineering lives. Scaling the shield exposed, in order: a sector-minimum filter that protected the sensor origin but not a hull with corners (v1); a hull-aware filter that deadlocked at spawn because its margin exceeded the true clearance (v2); a rotation livelock in a tight corner pocket that no forward candidate could escape (v4), fixed by a second “escape” tier (v5); and, only at campaign scale, an *absorbing freeze* in which a full stop won the escape tier's own scoring every tick and could never be unlatched (v7.4), fixed by a third “retreat” tier and a rule that the escape tier may never publish a stop while a certified mover exists (v7.5). We keep this chronology in the paper (Sec. IV-D) instead of presenting only the final filter. It is the evidence for a claim we would otherwise be asserting: that a structural no-contact guarantee is achievable at campaign scale, and that achieving it is a sequence of falsifiable fixes. The iteration-1 runs are kept as the diagnostic, the iteration-2 runs as the fix, and every dead shield version as an artifact.

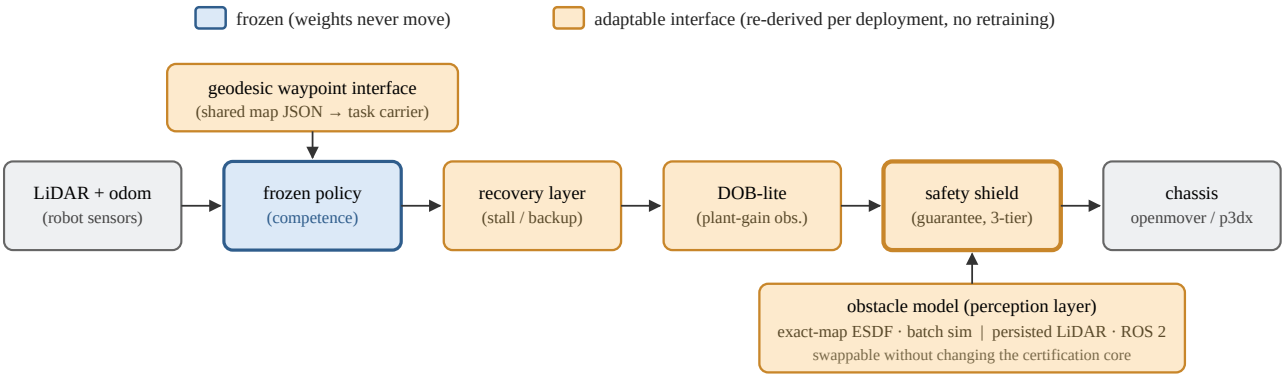


Fig. 1. The division of labour that survived every mechanism substitution, drawn as the system architecture. Exactly one block is frozen (ice-blue): the learned policy. Every other block is a mathematical interface re-derived per deployment (warm), including the shield's obstacle model, which swaps between an exact map-box distance field (batch simulation) and a persisted LiDAR grid (ROS 2 deployment) *without changing the certification core*. That swap is the modularity claim of Sec. IV-D in one picture. *Placement: full-width (figure*), page 2. Diagram drawn inline as SVG; regenerate as vector PDF for submission.*

IV. SYSTEM AND METHODS

Each component below is formalised *as it is actually implemented*. Where the framework draft named a more elaborate mechanism (a CBF-QP, a deployed Koopman controller, a full disturbance observer), we give the simpler object that was built and validated, and we say so. Kept source files are named inline so every equation below can be checked against code.

A. Source world and the frozen policy

The source task is point-goal navigation for a differential-drive robot (openmover: 2 × 600-bin LiDAR at ±90° front/rear, wheel radius 0.0504 m, separation 0.324 m) in 8 m rooms with 4–12 scattered walls, sampled so that every goal chord crosses at least one wall (blocked-goal sampling). Training followed 17 logged runs (≈50 M environment steps) in Isaac Lab [11], [12] with PPO [13]; the chronology and its negatives are in Sec. VI-A. The deployed policy is a behaviour clone of a validated geodesic expert under a corrected observation, with a contact-force channel: an MLP (1208 → 128 → 128 → 2) mapping a 1208-dimension observation (front/rear scans, goal bearing/distance, proprioception, geodesic waypoint and geodesic distance, and a saturating contact channel) to two wheel commands at 20 Hz. The checkpoint (`bc_init_v3/model_0.pt`) is frozen for every result in this paper.

B. Deployment rung and the shared world model

Deployment runs through plain ROS 2 topics [20] (Fig. 2): an Isaac Sim bridge publishes scan and odometry and consumes `/cmd_vel`; a policy node closes the loop. A single map JSON file is the shared world model: the bridge bakes its wall boxes into simulation colliders, and the policy node builds a geodesic field (GeoField) from the same boxes, exposing a waypoint and geodesic distance toward the goal as observation channels. Maps are authored by drawing a PNG and converting it through a BFS-validated pipeline (Fig. 3), so "a new world" costs one drawing, not one training run. Serial multi-goal tasks are composed at the interface: an ordered goal list advances on arrival and rebuilds the GeoField per leg, with zero weight changes.

C. Interface mechanisms

Geodesic waypoint interface (task/geometry axis). The GeoField waypoint is the task carrier across worlds; Sec. VI-B's blinding ablation shows it is load-bearing (removing it turns 4/4 successes into 0/1). It is the one interface that is present at *training* time as an observation channel and at *deployment* time as a field rebuilt from the target map.

DOB-lite (dynamics axis). A scalar plant-gain disturbance observer [15] (`dob_lite.py`) sits between policy and actuators and compensates a multiplicative thrust loss (a lower wheel-speed cap plus slip) that degraded dynamics introduce. It does not estimate a disturbance signal directly; it identifies the *plant gain* from realised wheel-joint speed and inverts it. For raw policy action a and gain estimate g, the shim applies a/g, so the realised wheel speed is $w_{real} = P \cdot a/g$ with P the true plant gain. Defining the per-step measurement and update:

$$q_t = mean_{wheels}\left(w_{real,t} / a_{t-1} \right) = P / g_{t-1} \tag{1}$$

$$g_t \leftarrow (1-\alpha)\, g_{t-1} + \alpha \cdot clamp\left(g_{t-1}\, q_t / W_{nom},\, 0.15,\, 1 \right) \tag{2}$$

with $\alpha = 0.10$ and nominal actuator gain W_{nom}. Because the target $g \cdot q / W_{nom} = P / W_{nom}$ cancels the current g, the fixed point is $g^* = P / W_{nom}$, which is *unbiased by the shim's own correction* (no algebraic loop). The source ($P = W_{nom}$) gives $g^* = 1$, a bit-exact no-op; the degraded case ($P = W_{deg}$) gives $g^* \approx 0.6$. The clamp to $[0.15, 1]$ means the interface only compensates loss, never over-boosts. Updates are gated to quasi-steady samples: $|a_{t-1}| > 0.05$ (deadband), $\mathrm{sign}(w_{real} \cdot a_{t-1}) > 0$ (sign consistency), and the command held near-constant for three steps (`settle_tol 0.08`); g resets to 1 on episode reset. The "mean g" column of Table II is the whole-run average of g, *including* the post-reset $g = 1$ steps, which is why it reads slightly above the converged value. Two earlier versions that observed *body* speed instead of wheel speed were rejected as biased ($g = 0.58$ and 0.485 on the *source* task, where the true answer is 1):

acceleration transients make realised body speed lag intended speed, so a body-speed observer degenerates into a speed booster. Moving the observable to the wheel joint, which tracks the actuator target with negligible inertia lag, is what makes Eq. (2) unbiased.

CBF-lite (safety axis, training-world concept check). A minimal directional-cone barrier (`cbf_lite.py`) used only for training-world A/B checks (Sec. VI-D); the deploy-time shield of Sec. IV-D supersedes it. It is a *closed-form clamp, not a QP*. With `d` the LiDAR clearance inside a ±45° cone around the travel direction and `v` the intended body speed, the barrier and its safe-speed bound are

$$h(d,v) = (d - d_{margin}) - v^2 / (2\,a_{dec}), \quad v_{safe}(d) = \sqrt{(\,2\,a_{dec}\,\max(d - d_{margin}, 0)\,)} \tag{3}$$

with d_{margin}=0.30 m (expressed at the LiDAR origin, 0.22 m ahead of the base, so it covers the 0.327 m hull half-diagonal at any yaw) and a_{dec}=1.0 m/s². Only the forward component is clamped to $|v| \le v_{safe}$; the spin component is left untouched, so the policy can always turn in place to find a clearer heading. It simply cannot drive into a wall faster than it can stop. When the clamp never fires the shim returns the input tensor unchanged, reproducing the frozen baseline bit-for-bit. There is no $\dot{M}h$, no $\alpha(h)$ and no solver: this is the stoppability bound of Eq. (3) evaluated directly. An earlier hemisphere-minimum variant never fired once in 9620 steps (the bound was dominated by corridor side walls the robot was driving *past*), which is why the clearance is measured only in the travel cone.

Koopman pipeline (dynamics model, offline only). A frozen extended-dynamic-mode-decomposition (EDMD) lift (`30_koopman_fit.py`) of the state $s=(v_x,\ w_z,\ w_l,\ w_r)$ into a 12-dimensional observable vector, a deliberate *subset* of the full 15-term quadratic that omits the three cross terms $v_x w_r$, $w_z w_l$, $w_l w_r$:

$$\psi(s) = [\ 1,\ v_x,\ w_z,\ w_l,\ w_r,\ v_x^2,\ w_z^2,\ w_l^2,\ w_r^2,\ v_x w_z,\ v_x w_l,\ w_z w_r\ ]^T \tag{4}$$

A control-augmented lifted-linear model $\psi(s_{t+1}) \approx \Theta\,[\psi(s_t);\ u_t]$ with $\Theta \in \mathbb{R}^{12\times 14}$ is fit by ridge regression on inputs standardised by their training statistics ($\lambda=10^{-3}$; the unlifted affine baseline uses 10^{-4}), with the prediction target kept in natural lift units:

$$\Theta = \mathrm{argmin}_{\Theta}\ ||\ \Psi' - \Phi_{std}\,\Theta^T\ ||^2 + \lambda\ ||\Theta||^2 \tag{5}$$

The protocol fits on source transitions, evaluates one-step prediction on degraded data with the *frozen* source fit (the transfer gap), then refits on degraded data alone (the adapted bound), against persistence and affine baselines. We stress the scope: this is a *pipeline proof-of-existence, not a controller*, and the lift is never closed around the plant anywhere in this paper. Standardisation is not cosmetic. The first unstandardised fit diverged to $R^2 \approx -10^6$ on v_x because the raw quadratic features span $10^0 \ldots 10^3$. The result (Sec. VI-C) is that the quadratic lift barely beats the affine fit at this scale; what transfers is the freeze-structure/adapt-interface pattern, not the lifting mechanism.

D. The deploy-time safety shield

The shield is the only component with final authority: it sits between the policy and the actuator, the frozen policy is deployed *unchanged*, and when no command is filtered the shim returns its input tensor untouched, reproducing the unprotected baseline bit-for-bit. Everything below is one certification core (identical constants, identical tier logic) instantiated against two different obstacle models (Sec. IV-D3).

1) Certification core. The hull is the 0.44×0.28 m footprint (half-extents h_x=0.22, h_y=0.14, half-diagonal 0.327 m), sampled by 51 points: the centre plus 50 perimeter points at 0.03 m spacing (15/10/15/10 along the four edges). The spacing is a correctness requirement, not a tuning knob, and we learned it the expensive way. The v6 shield used 17 hull points (corners plus edge mid- and quarter-points), which left 0.11–0.22 m gaps along the long edge, the same order as the 0.125 m pallets and wall ends in the deployment maps. A thin box could sit entirely *between* two samples while the hull rotated into it, and it did: v6 penetrated by −0.0405 m and −0.0564 m on the two gate maps, both times by the hull corner (0.22, ±0.14) into a 0.12 m-thick box (Sec. VII-B). At 0.03 m spacing the perimeter discretisation costs at most spacing/2 = 0.015 m, which the certified slack pays for explicitly.

Each candidate twist (v,ω) is forward-simulated as a constant-curvature arc for HORIZON = 0.8 s at DT = 0.05 s (16 steps), followed by a straight braking tail that decelerates at BRAKE_A = 0.6 m/s² with ω=0 (up to 17 steps), giving K = 33 poses per candidate. The tail is what makes this a *stoppability* test rather than a reachability test: a command is certified only if the hull stays clear through the manoeuvre *and* through the stop that follows it. With $d(\cdot)$ the signed distance to the obstacle model (negative inside an obstacle) and p_k the arc pose at step k, the clearance of a candidate and its admissibility at margin m are

$$c(v,\omega) = \min_{k=1..K} \min_{h=1..51}\ d(\,R(\theta_k)\,q_h + t_k\,), \quad \text{admissible} \Leftrightarrow c(v,\omega) \ge m + \sigma \tag{6}$$

where q_h are the hull samples and σ = ESDF_SLACK = 0.05 m is an explicit optimism budget: it bounds the bilinear sampling error of a 1-Lipschitz field on the 0.025 m lattice ($\le$ res√2 = 0.035) plus the hull perimeter discretisation ($\le$ 0.015). A candidate accepted at $m+\sigma$ on the sampled field therefore really does keep the true hull at least m off the true obstacles. We verify that claim numerically instead of asserting it (Sec. IV-D4).

The command set is the grid $v \in \{-0.2,-0.1,0,0.1,0.2,0.3,0.4,0.5\} \times \omega \in \{\pm 0.9, \pm 0.6, \pm 0.3, 0\}$, that is 56 nominal candidates. Tier 1 first tests the policy twist itself and then the last published twist (two arcs, 2×33×51 = 3366 field samples); if neither is admissible it projects onto the safe subset of the full grid:

$$(v^*,\omega^*) = \mathrm{argmin}_{c(v,\omega)\ge m+\sigma}\ [\,(v_c - v)^2 + (\,0.25\,(\omega_c - \omega)\,)^2\,] - 0.05\,\mathbb{1}_{[\text{repeat}]} \tag{7}$$

The `0.25 m/(rad/s)` weight (`W_SCALE`) converts yaw rate into the tangential speed it imposes on the 0.22 m hull corner; the `0.05` bonus (`HYST_BONUS`) makes repeating the last published twist beat a marginal win, which is what stopped a 20 Hz sign-flip chatter that v6 exhibited (`+0.10 ↔ −0.10 m/s` published while the chassis realised `≈0.01 m/s`).

Eq. (7) is exactly the objective of a CBF-QP (minimal deviation from the nominal command subject to a forward-invariance constraint), solved by enumeration instead of by a solver. Two consequences matter. First, *the filter carries no task objective of its own*: there is no progress term in Eq. (7). Goal-directedness enters only through `backup()`, which is invoked when the projection would freeze (the winner is a stop, or $|v_c|<0.1$, or the pin latch is held) and which selects, *within the already-certified subset*, the arc maximising

$$g = t_K \cdot \hat{u} + 0.15\cos(\theta_K - \mathrm{atan2}\,\hat{u}) \tag{8}$$

with $\hat{u}$ the unit robot-frame direction to the geodesic waypoint (Sec. IV-B). Safety is never traded against progress; progress is only ever chosen among safe options. Second, the stop command `(0,0)` is not axiomatically admissible. It is tested by Eq. (6) like every other candidate, because a robot already inside the certified floor cannot be made safe by standing still. This is why a third tier is needed, and it corrects an earlier draft of this work that claimed “the worst case is standing still”.

2) Three tiers. The cascade of Fig. 2 has three certified-or-better levels with strictly decreasing floors.

- **Tier 1, comfort** (`m = MARGIN = 0.10`, floor `0.15 m`). Pass-through, then the projection of Eq. (7) over the grid, then the `backup()` freeze guard of Eq. (8).
- **Tier 2, escape** (`m = ESCAPE_MARGIN = 0.02`, floor `0.07 m`, still *certified*). Engaged only when tier 1 admits nothing but a stop: in a tight corner pocket the comfort envelope can outrun the clearance the hull actually keeps, and the robot livelocks in place rotation (observed at tour goal 3, v4). Candidates are capped at `ESCAPE_VMAX = 0.20 m/s`, the hull is grown by `RAW_PAD = 0.08 m` for the raster half of the barrier, and the projection target is flipped to a *reverse* speed `−max(0.10, min(|v|,0.20))` whenever the pin latch is held, so a robot that backed into a pocket backs out of it. Since v7.5 the escape tier never publishes a stop while a certified mover exists: if both the projection and `backup()` return `(0,0)`, the selection is restricted to certified non-stop candidates and re-taken. Without that rule the hysteresis bonus on a published stop made the stop win every tick. That is an *absorbing freeze*, because the pin latch releases only on 0.20 m of motion that never comes (v7.4, found at campaign scale on `f1_b0_m01`).
- **Tier 3, retreat** (*uncertified*). Reached only when even the 0.07 m floor admits no mover, i.e. when the hull is already inside the certified envelope and no command can restore it. It maximises clearance gain $c_{end} + c_{min} + 0.02|\omega|$, where the clearance gradient is itself the direction signal, and when the pin latch is held it forbids candidates with $|v| \le 0.03$ so that the robot must translate and the latch can release. Tier 3 deliberately carries no continuity term. Adding the hysteresis bonus here (v7.6) stopped a 20 Hz dither and fixed one scrape, but the bonus is a lock-in precisely where nothing is certified, and it created two *new* scrapes including one on the thesis arm; the change was reverted after validation. Net-stationary dither is empirically safer than committed uncertified motion.

The **pin latch** that tiers 2 and 3 consult is purely geometric: 10 consecutive vetoed ticks (0.5 s at 20 Hz) with less than `0.03 m` of motion latch `pinned`; `0.20 m` of real motion releases it. While pinned, tier-1 pass-through is *suspended*, since otherwise certified-safe micro-twists from the policy abort the backup policy's re-aim every other tick. An earlier stall detector based on a chassis-speed proxy was defeated by the v6 chatter (it reported `pinned = 0` in 13 of 14 fortress escapes, so the reverse-preferring escape never engaged and the robot dithered for `≈80 s` at +0.05 m clearance); the geometric latch does not depend on that proxy.

Two smaller interface policies complete the node and are labelled as such, not as learned behaviour. **park-after-success**: the frozen policy has no stop-at-goal behaviour and grinds into the goal pocket if left running, so the node publishes zero twist after the final goal; this was a dominant component of iteration-1 contact. And an **out-of-bounds ramp** in the distance field (`−(1+depth)` instead of a constant `−1`), because a constant leaves every consumer without a gradient once the hull crosses the map edge: all candidates tie, and tier 3's tie-breaks then drive the robot *further* out.

3) Two obstacle models, one certification core. Nothing in Eqs. (6)–(8) refers to a sensor or to a map. The only per-deployment input is the signed-distance oracle `d(·)`, and the benchmark exercises two of them with byte-identical tier logic and constants (Table I). This is the modularity claim made concrete: the guarantee arm is decoupled from the perception layer, and it holds whether the oracle is fed a flawless exact-map field (high-throughput batch simulation) or a noisy, persisted LiDAR grid (hardware-realistic ROS 2 deployment). The two differ in what the guarantee is *relative to*, exact geometry in one case and sensed evidence in the other, and we report both readings separately.

Table I. The two obstacle-model stacks behind one certification core. Tier logic and all `SHIELD_*` constants are identical; only the signed-distance oracle differs.

	Stack A: `shield_lite.py` (v7.5)	**Stack B: `policy_node.py` (v7.4)**
oracle `d(·)`	exact signed distance to the map's wall boxes (`BoxSDF`), true distance on a 0.025 m lattice, bilinear sampling, clamped at +1.0 m halo per box	union of (i) the same exact map-box field and (ii) a conservative boolean raster of *persisted* LiDAR hits, 0.08 m cells, ±2.6 m robot-frame extent, dilated by a 2-cell Euclidean disk
memory	none (the map is the whole world)	hits are never cleared: the map is static, and a wall closer than the scanner's 0.22 m near clip drops out of the scan; without persistence the raster barrier goes blind to it
unmapped-hit filter	inert: every obstacle is a map box and the raycaster is exact, so all hits are explained by the map and the raster half would be empty	a hit whose exact-field value is ≤0.05 m is dropped (it would be double-counted 4–9 cm pessimistically); anything the map does not know about (a moved crate, a person) survives into the raster
sensors	exact raycaster	two planar LiDARs at hull-frame `x = ±0.22015 m`, yaw 0 and π
candidate set	48 of the 56, filtered to the wheel-realizable simplex $\lvert v\rvert/v_{full}+\lvert\omega\rvert/\omega_{full} \le 1$	all 56 (the bridge chassis realizes the published twist directly)
action path	wheel fractions in $[-1,1]^2$, $v_{full} = 0.5035$ m/s, $\omega_{full} = 3.108$ rad/s	`/cmd_vel` twist
guarantee relative to	**exact geometry**	**sensed evidence** (and exact geometry where sensed)
used for	the 708-episode campaign (Sec. VI-G) and the held-out endgame (Sec. VI-H)	the ROS 2 presentation runs (Sec. VI-E) and both embodiment plants

The realizability filter in Stack A is a porting subtlety worth stating because it is a safety issue, not an optimisation. There the shield emits *wheel fractions*, so a certified `(0.5, 0.9)` would be silently shrunk by actuator saturation into a twist whose arc was never checked. Dropping unrealizable pairs removes only commands the plant cannot follow.

4) Is the certified floor actually certified? A margin is only a guarantee if the quantity the shield checks lower-bounds the quantity an independent audit measures. We check this offline (`32_check_shield_v7.py`: no simulator, no ROS, the barrier methods exercised against brute-force exact point-to-box distances) on the two maps where v6 had failed, over 2850-frame traces each.

- **Field vs exact.** Max over-read of the bilinear lattice field against brute-force distance on ≈4000 in-halo points per map: `+0.0125` / `+0.0123 m` (budget $\sigma = 0.05$).
- **Hull discretisation.** The 51-point hull against a 361-point reference hull (0.004 m spacing) over every frame: max over-read `+0.0066` / `+0.0068 m` (budget `spacing/2 = 0.015`).
- **Total optimism** `+0.0066` / `+0.0124 m` ≤ 0.05, so the verdict is **CERTIFIED** on both maps, with ≈3× headroom.
- **Replay.** Of the frames whose next 0.8 s of *real* trajectory breached the certified floor (1644 / 1896), tier 1 vetoes 1617 / 1823. The 27 / 73 misses are margin misses, not collisions: their true future minimum is `+0.085`…`+0.121 m`, just under the 0.15 m floor, attributable to the finite-difference twist of the recorded plant differing from the arc the barrier simulates. Every frame in which the hull was truly inside a box was vetoed: 53/53 and 395/395, zero missed.

Field *under*-reads (up to −1.0 m outside the padded grid) are conservative by construction: they cost admissible candidates, never safety.

5) Cost. Measured on CPU against the 20 Hz budget of 50 ms: `0.31 ms` for the two-candidate pass-through check that runs every tick, and `3.62` / `3.59 ms` for the full 56-candidate batch that runs only on a vetoed tick, i.e. roughly 7 % of the budget at the worst case, with ≈14× headroom on the common path. An earlier draft of this work quoted "≈14 ms per intervention tick"; that number was never measured and we withdraw it in favour of the figures above, which come from the validation script named here.

6) Guarantee statement. With the shield active, no command reaches the chassis unless its 51-point hull, forward-simulated over 0.8 s plus a braking tail, keeps at least `0.15 m` (tier 1) or `0.07 m` (tier 2) of clearance to the obstacle model at every one of 33 poses; when neither is achievable the command comes from tier 3, which is *explicitly uncertified* and is reported separately. In the thesis arm of the campaign it fired 5327 times out of 452 404 env-steps (1.2 %), itself a subset of 12 684 escape-tier firings (Sec. VI-G). Formally: the certified set of Eq. (6) is forward-invariant under bounded disturbance *relative to two models*, the obstacle model (exact geometry in Stack A, sensed evidence in Stack B) and the hull model. The second relativisation is not hypothetical. In the counterexample analysis of Sec. VII-A the certified hull never penetrated (min `+0.070 m`, 0/2711 frames) while the simulator's contact sensors did fire, because the drive wheels sit at `y = ±0.162 m`, which is `2.2 cm` outside the $h_y = 0.14$ hull, and the casters sit outside it too. The fix is a wider hull box, not a different filter. We report the gap and not the cleaner number.

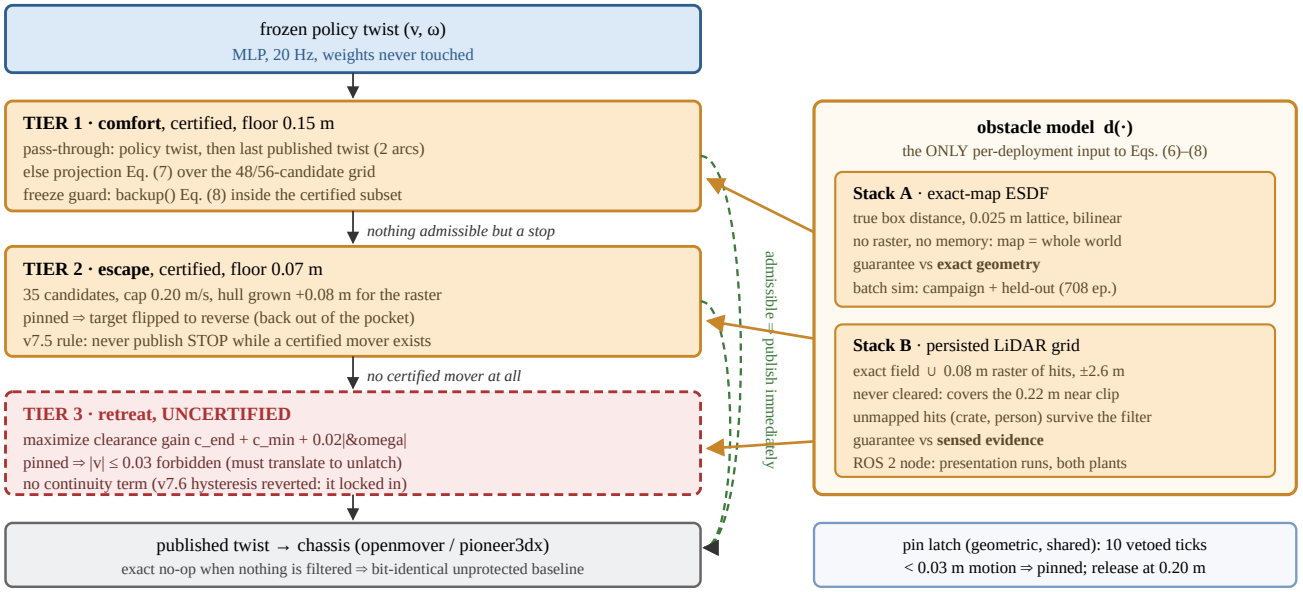


Fig. 2. The shield's decision procedure. Three tiers with strictly decreasing clearance floors sit on *one* certification core, Eq. (6) over a 51-point hull forward-simulated for 0.8 s plus a braking tail, and the only per-deployment input is the signed-distance oracle on the right. Tiers 1 and 2 are certified (0.15 m and 0.07 m); tier 3 is not, is drawn dashed and red for that reason, and is counted separately in every results table. Green dashed arcs are the publish shortcuts: a tier that finds an admissible command publishes it immediately and the lower tiers never run. Swapping the oracle between Stack A and Stack B changes what the guarantee is relative to (exact geometry vs sensed evidence) without changing a single line of the tier logic. *Placement: full-width (figure*), page 4. Diagram drawn inline as SVG; regenerate as vector PDF for submission.*

V. EXPERIMENTAL SETUP

A. Simulator, plants, protocols

Everything runs in Isaac Lab / Isaac Sim [12] on a single workstation (RTX 3080, 16 GB RAM). The deployment rung is ROS 2 Jazzy [20] over the Isaac Sim bridge, and only one simulator instance runs at a time. The plant is a differential-drive robot with wheel separation `0.324 m`, wheel radius `0.050353 m` and a wheel speed limit of `10 rad/s`, which gives `0.5035 m/s` body speed. The simulation step is `1/120 s` with decimation 6, so control runs at 20 Hz. The action is two wheel-velocity targets in `[-1,1]`. The observation has 1208 dimensions: two 600-ray planar scans (bluesea2 pattern, ±90°, 10 m range, updated at 15 Hz), the goal vector, body twist, the 3-dim geodesic-waypoint channel and one normalised net-contact-force channel. Episode horizons in the source world are 15 s. Deployment horizons are derived from the map and are never tuned per map:

$$T_{horizon} = min(\ len(\gamma_{geo}) / 0.25 + 10,\ cap\ )\ s \tag{9}$$

Here γ_{geo} is the inflated geodesic path. For the 34.38 m held-out map of Sec. VI-H this is `147.5 s`.

The study uses three map sets, and they are disjoint. (i) *Source worlds* are the procedural training distribution. (ii) *Presentation maps* are user-drawn and converted by the pipeline of Fig. 3: *demo_open* (scattered walls), *demo_mid* (full-width staggered partitions, ≈2 m gaps), *demo_gauntlet* (same partitions, gaps offset ±0.8–0.9 m off the direct line), *demo_crossings* (rail-yard gaps on the spawn–goal line) and *demo_tour* (three ordered goals), plus a difficulty ladder (easy/mid/hard) that holds spawn, goal and the 13.2 m BFS path fixed and changes only geometry. (iii) The *campaign set* holds 56 procedural maps from 7 families (corridor, gates, loop, fortress, competition, warehouse, highway) × 4 difficulty bins × 2 variants, produced by a seeded, BFS-validated generator with per-family motif counts recorded in a manifest.

Two campaign maps (`f4_b1_m00`, `f6_b2_m00`) are locked as the demonstration pair and run 10 spawn-yaw seeds instead of 4, giving 54×4 + 2×10 = 236 episodes per arm and 708 across the three arms. Every (map, arm) cell runs in a fresh simulator process at torch seed 42 with the spawn-yaw repeats folded into parallel envs. One script (`33_run_campaign.py`) orchestrates the campaign end to end and records per-run wall clocks, return codes and episode-capture completeness. The held-out map of Sec. VI-H belongs to none of the three sets.

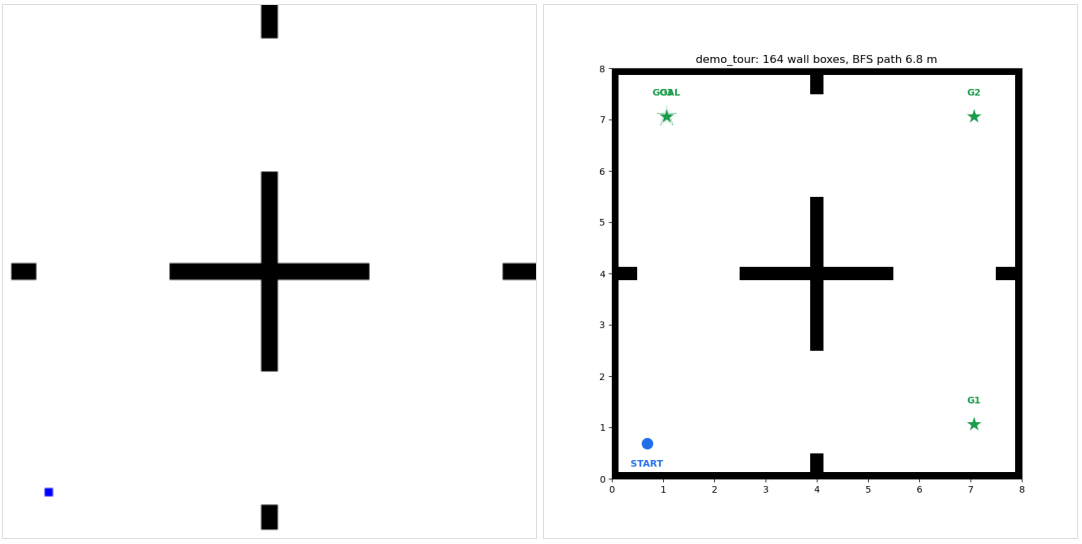


Fig. 3. The map pipeline: a user-drawn PNG (left) is converted to a BFS-validated wall-box JSON and rendered back for inspection (right). The same JSON feeds the simulator colliders, the policy's geodesic field and the shield's exact-box distance field, so one file serves three consumers and nothing is re-annotated. *Placement: full-width (figure*), page 5. Source images: openmover_ws/isaac/lab/maps/demo_tour{,.preview}.png.*

B. Metrics, and two different contact instruments

Contact is reported with two instruments that measure different things, and we keep them separate everywhere in this paper.

The first instrument is geometric penetration, measured by the exact-box audit. `30_clearance_audit.py --map_file` transforms the hull sample points of every recorded trace frame into world coordinates and evaluates exact point-versus-box signed distance against the map's wall boxes, the same boxes the bridge bakes into colliders. A negative value means the hull is inside a wall. The default (raster) mode uses the 0.125 m occupancy field and over-flags: it reports a hull point as penetrating whenever the point lies in an occupied *cell*, even if the baked wall surface is on the far side of that cell. Every count in this paper is exact-box and recomputable from the kept traces.

The second instrument is physical contact, the >5 N latch. The campaign and held-out arms report `collision` as the environment's own latch on a simulator contact force above 5 N on the base link. Collision is a per-step *penalty* and not a terminal condition, so an episode can collide and continue. This is the instrument behind “0.534” and “0.000” in Tables V and VI.

The two instruments can disagree, and when they do the disagreement is informative. Sec. VII-A documents an episode whose certified hull box never penetrated (min `+0.070 m`, 0/2711 frames) while the >5 N latch fired, because the drive wheels and casters sit outside the certified box.

The audit hull is coarser than the shield hull. The audit samples the `0.44×0.28 m` hull with 13 points: the centre, the four edge midpoints, the four corners, and four quarter-points at `(±0.22, ±0.07)`. That leaves 0.22 m gaps along the long edges and 0.07 m along the short ones, so the audit is coarser than the 51-point hull the shield certifies against, and a reported audit clearance is an upper bound on the true hull clearance: a very shallow graze landing entirely between two audit samples would not be flagged. An earlier draft of this work described the audit hull as 17 points; the script has 13. Three things bound the residual risk. The shield's own certification uses the dense 51-point hull and was validated against a 361-point reference hull (Sec. IV-D4). The audit caught every real contact we produced, down to `−0.003 m`. And the frame-exact numpy audit of Sec. VII-A is an independent implementation. Counts are reported both for the full recorded run and for the *approach* phase, up to first arrival within the 0.30 m success radius of the final goal, because the unshielded policy's post-success grind is a different behaviour from approach contact and pooling the two would flatter neither.

The remaining metrics are defined as follows. *Blocked-goal success* (source world) is the fraction of episodes reaching within 0.30 m of a goal sampled with a wall guaranteed between spawn and goal, under deterministic mean-action inference, seed 42, 256 envs × 2405 steps (≈2.0–2.1k episodes per world). *TTG* is mean time-to-goal over successful episodes only. *Shield interventions* are ticks on which the published command differs from the policy command, reported as a percentage of env-steps and broken into escape-tier and retreat-tier firings wherever tier 3, the uncertified tier, could have contributed. *oob* counts env-ticks spent outside the barrier's map-rectangle domain, an off-model audit that should be zero and is.

C. Reproducibility

Traces, logs, maps, manifests, checkpoints and scripts behind every number are kept in the project workspace, and figures are regenerated from those artifacts by `draft/make_figures.py`. Negative artifacts are kept alongside positives: the deadlocked shield traces (v2 spawn deadlock, v4 tour livelock, v7.4 absorbing freeze), the v6 penetration traces that forced the exact distance field, the corrupted-waypoint runs whose conclusions were invalidated, the wedge-failure traces, the blinded-ablation traces, and the `num_envs=1` counterexample of Sec. VII-A. Where a result was superseded, the superseded numbers remain in the ledger instead of being edited out.

VI. RESULTS

The subsections follow the two arms of the thesis. Sec. VI-A reports what the learned arm cost to obtain and what it could not do. Secs. VI-B and VI-C follow the frozen policy across the geometry, dynamics and embodiment axes. Secs. VI-D and VI-E follow the guarantee arm from a training-world concept check to a zero-contact deployment iteration. Secs. VI-F, VI-G and VI-H put the two arms side by side: first in a consolidated transfer ledger, then in a 708-episode three-arm campaign, then on a held-out map drawn by a person.

A. What training bought, and what it killed

The training chronology is part of the result, and most of it is negative. PPO [13] from scratch never bootstrapped blocked-goal reaching: the best of the early runs reached 0.095 blocked success, and neither curriculum nor clearance shaping fixed it, so exposure was not the binding constraint. Reward-space geodesic shaping made it *worse*, and observation-space waypoint injection alone produced a cautious in-place wiggle. Both of those conclusions were later found to be confounded by a corrupted waypoint observation: auditing 6232 sampled waypoints showed 40 % of the straight-line chords passing through walls and 30 % of the waypoints lying inside wall geometry. The bug invalidated the conclusions of five runs (run12–14 and their shaping variants), and we report them as invalidated instead of re-interpreting them after the fact.

Behaviour cloning from a validated geodesic expert under the corrected observation reached 0.234, and DAgger [10] relabelling gave 0.224, so the gap did not close. Distribution shift is provably *not* the binding constraint in this pipeline, which means the standard remedy for imitation learning does not apply here. Adding the net-contact-force channel to the observation gave the final 0.287 blocked success (run17, checkpoint `bc_init_v3/model_0.pt`), against a geodesic expert at 0.636 and a drive-straight floor of 0.036. The expert could see the force it was recovering from and the student could not, which is where cloning stalled. The frozen policy is thus ≈8× the trivial floor and ≈45 % of its own expert. Two readings matter for the thesis: competence required a mathematical interface (the geodesic waypoint channel) to exist at all, and most of the compute spent bought falsified hypotheses instead of improvements.

B. Geometry axis: a gradient with a measured boundary

With the weights frozen, the policy solves user-drawn maps it never saw: crossings in 33.5 s, ladder-easy in 29.3 s, ladder-mid in 26.2 s, and a three-goal tour in 46.1 s (Fig. 4). The mid rung has full-width partitions, a map class absent from training, and it is solved *faster* than easy (26.15 vs 29.30 s), because the partitions funnel approaches toward the gaps the policy already knows how to thread. Transfer therefore tracks behavioural overlap with the source distribution rather than visual complexity, and the ladder measures that overlap. Two failures bracket the gradient: the hard rung does not finish, wedging 4.62 m short at the second gap corner `(4.2, 3.85)` after 25 recoveries, because the alternating backup recovery cannot change the approach angle enough; and a serial-rooms map with 1.0 m doors is 0/3. Four seeded repeats with spawn jitter (±0.15 m, ±15°) succeed 4/4 in 20.9–33.5 s, so the positive side of the gradient is not a lucky seed.

The blinding ablation isolates the carrier. Zeroing the geodesic-waypoint and geodesic-distance channels on a map where the same checkpoint succeeds 4/4 turns it into 0/1, with the robot pinning 5.74 m short against the first wall. The frozen LiDAR and contact reflexes do not compose into gap-threading on their own; the mathematically derived channel is load-bearing. Under this division of labour the learned weights carry the reflexes and the interface carries the task.

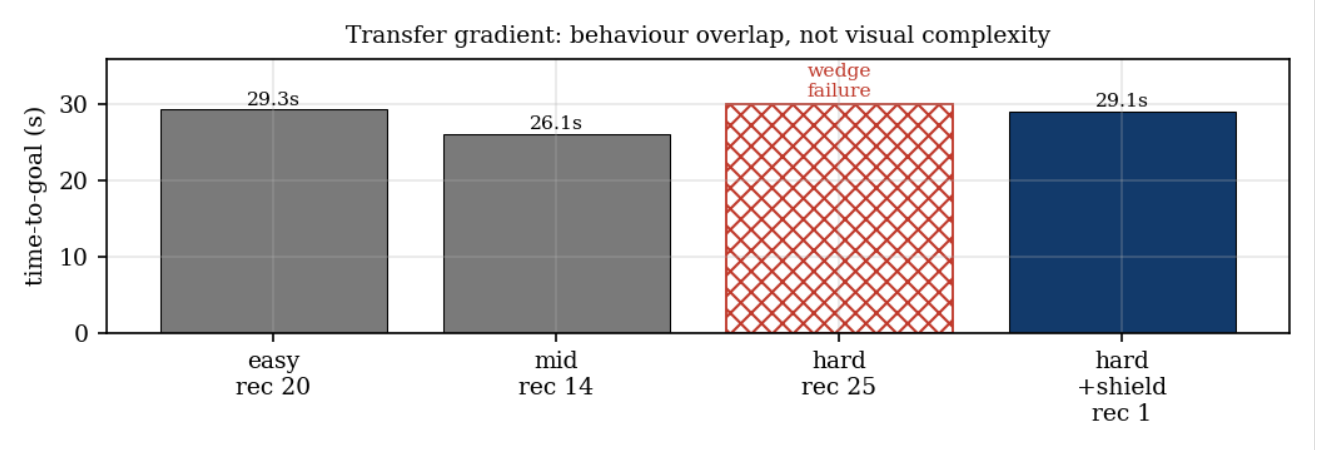


Fig. 4. Difficulty ladder, frozen weights: identical spawn, goal and 13.2 m BFS path; only the geometry changes. Easy and mid succeed, and mid is *faster* than easy because of behavioural overlap rather than visual complexity; hard wedges 4.62 m short at the second gap corner. Adding the deploy-time shield (Sec. VI-E) converts the hard rung into a 29.1 s success with zero penetration frames and one recovery. Recovery counts below the axis. *Placement: full-width (figure*), page 6. Generated by make_figures.py from ledger §13/§15.*

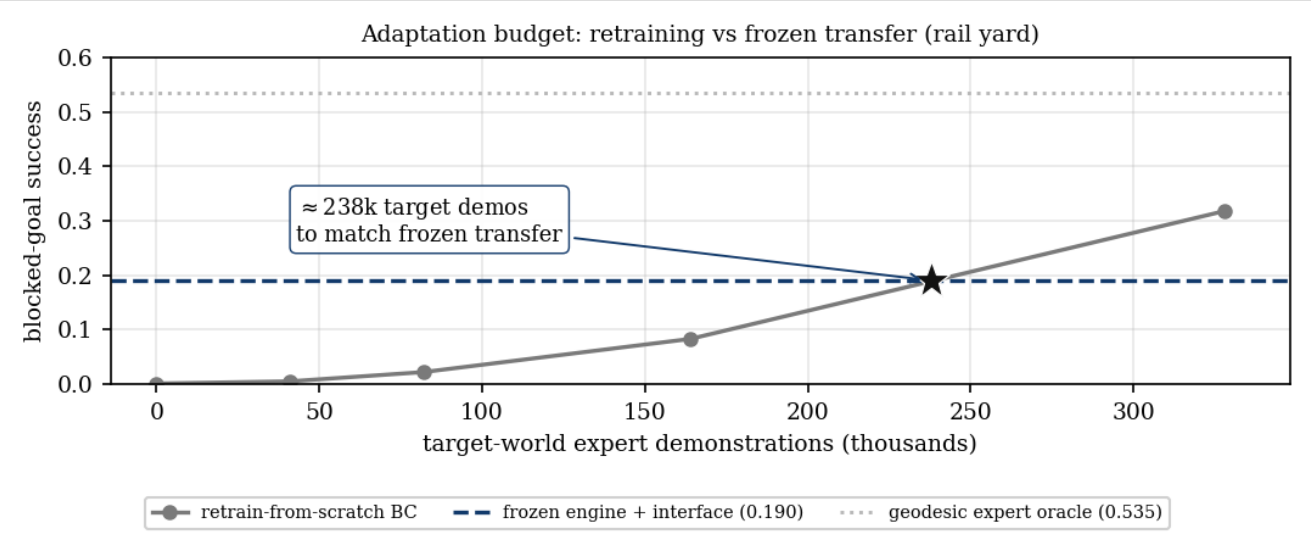


Fig. 5. Adaptation budget on the rail-yard geometry axis: blocked-goal success of from-scratch behaviour cloning against the target-world expert-demonstration budget (0 → 0.001, 41 k → 0.005, 82 k → 0.022, 164 k → 0.083, 328 k → 0.318), against the frozen engine's zero-shot 0.190 (dashed) and the geodesic expert oracle 0.535. Linear interpolation between 164 k and 328 k places the crossing at ≈238 k demonstrations. Colours follow the paper-wide grammar: grey = no mathematical interface, blue family = interface active (navy dashed = the frozen engine plus interface at zero target data). The caveats all push in the same direction and are printed here: BC is the most sample-efficient from-scratch baseline available to us, so 238 k is a *lower* bound on what retraining would cost; the curve has not saturated at 328 k; one seed per point. *Placement: full-width (figure*), page 6. Data: ledger §6.*

The adaptation-budget curve (Fig. 5) converts the transfer claim into an exchange rate: matching what the frozen engine delivers with no target data at all would cost on the order of 238 k expert demonstrations in the target world. This is the number we set against retraining from scratch, with the caveats above printed next to it rather than in a footnote.

C. Dynamics axis: the observer adapts, the weights stay frozen

Under degraded actuator gain (thrust ×0.6) the frozen policy's blocked success collapses from 0.287 to 0.028, a 90 % loss from a change the network was never told about. Adding DOB-lite, with no adaptation data and no weight change, restores 0.363, which is *above* the nominal frozen baseline, and the identified gain converges to `g* = 0.599` against the true 0.6, so the interface discovers the degradation factor from runtime data alone (Eq. (2)). An oracle arm told the factor in advance reaches 0.375, which puts the online observer within 3.2 % of an upper bound it never sees. On the source task, where the true answer is `g=1`, the observer converges to 0.964 and costs nothing (0.287 → 0.303); it is not a booster. Under combined shift, meaning degraded dynamics *and* unseen geometry together, it repairs the dynamics half (0.044 → 0.250, `g=0.602`) while the geometry half still costs. One interface does not fix both axes, and none is claimed to (Fig. 6, Table II).

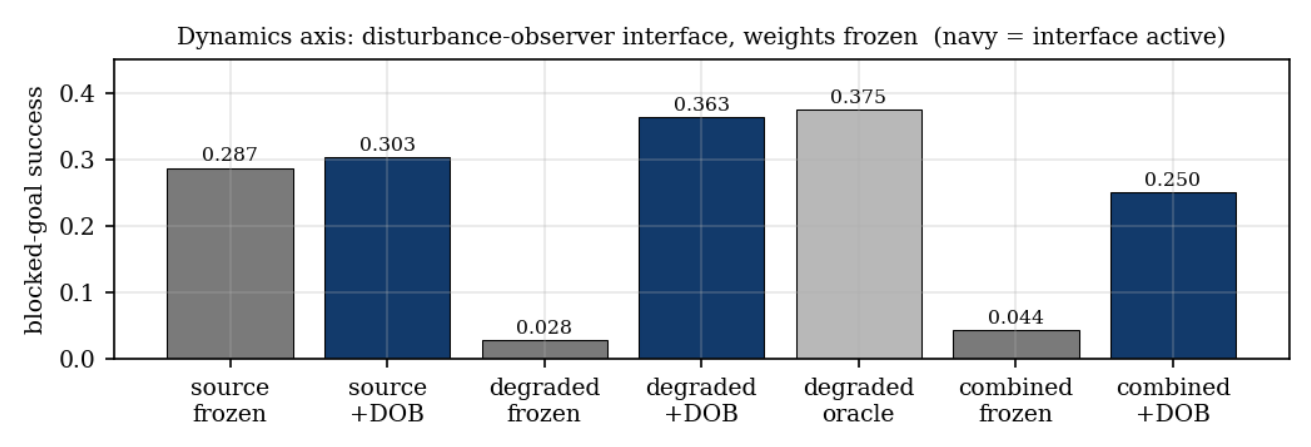


Fig. 6. Dynamics-axis A/B, frozen weights throughout. Degraded: 0.028 → 0.363 with the observer, against 0.375 for an oracle that knows the degradation in advance (within 3.2 %). Combined shift: 0.044 → 0.250. Source: 0.287 → 0.303 with the observer converging to g = 0.964 where the truth is 1. *Placement: full-width (figure*), page 7. Data: ledger §7.*

Table II. DOB-lite A/B under a frozen policy (blocked-goal success; ledger §7). "mean g" is the whole-run average of the observer state including post-reset `g=1` steps, so it reads slightly above the converged value quoted in the text.

deployment	blocked succ.	collision	mean g
source, frozen	0.287	0.642	—
source, +DOB-lite	0.303	0.644	0.964
degraded (×0.6), frozen	0.028	0.472	—
degraded, +DOB-lite	**0.363**	0.617	0.599
degraded, oracle boost	0.375	0.629	—
combined (degraded + unseen geometry), frozen	0.044	0.719	—
combined, +DOB-lite	0.250	0.798	0.602

The Koopman concept check mirrors this on the model side (Fig. 7). A frozen source-world lifted fit keeps the body-twist channels usable on degraded data (R^2 0.811 on v_x, 0.758 on w_z) but loses the wheel channels: 0.88 / 0.88 on source-world validation falls to 0.565 / 0.567, exactly where the degradation acts, and drops *below* the persistence baseline (0.790 / 0.773) there. On those channels the frozen lift fails. A small refit of the lift statistics on degraded data alone recovers 0.896 / 0.884 / 0.992 / 0.989. Two negatives accompany this result: the quadratic lift barely beats a plain affine fit at this scale, and the lift is never closed around the plant anywhere in this paper. What transfers is the freeze-structure/adapt-interface pattern rather than the lifting mechanism.

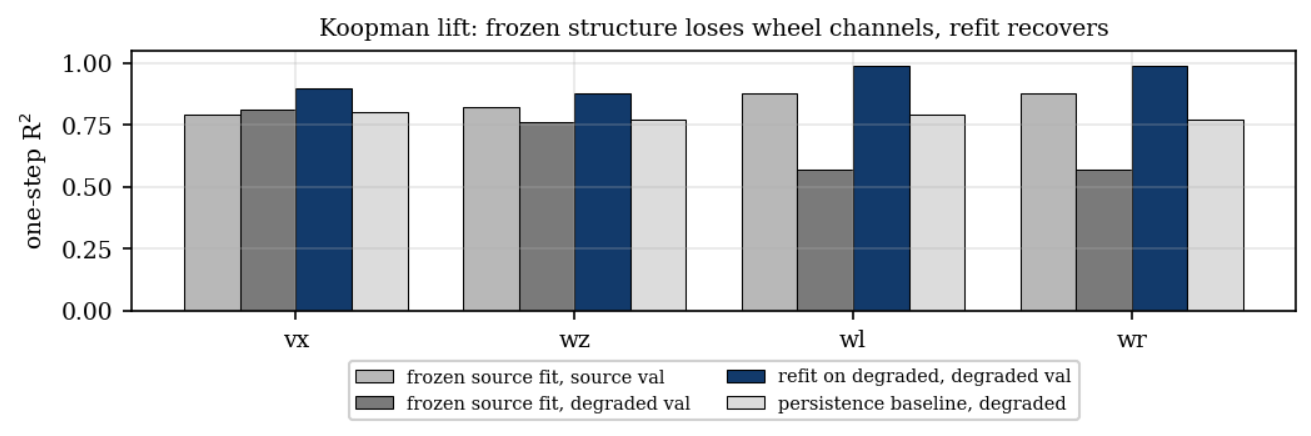


Fig. 7. One-step prediction R^2 per state channel $[v_x, w_z, w_l, w_r]$ on degraded data. Frozen source lift [0.811, 0.758, 0.565, 0.567]; persistence baseline [0.796, 0.770, 0.790, 0.773]; refit on degraded data [0.896, 0.884, 0.992, 0.989]. The wheel channels fall below persistence under the frozen lift, so the failure is where the degradation acts. *Placement: full-width (figure*), page 7. Data: ledger §11.*

D. Safety axis, part 1: a minimal cone barrier and the no-op that motivated the shield

The shield of Sec. IV-D was not the first safety interface we built, and the difference between the two is measured. The concept check (`openmover_lab/cbf_lite.py`, runner flag `--cbf_lite`) instantiates Eq. (3) in its smallest useful form: a single directional cone clamps the forward command to the stoppable speed $v_{safe}(d) = \sqrt{2\, a_{dec}\, \max(d - d_{margin}, 0)}$, where d is the LiDAR clearance inside ±45° of the *actual travel direction* (front scan for $v \geq 0$, rear scan for $v < 0$), $d_{margin} = 0.30$ m is measured at the sensor origin, and $a_{dec} = 1.0$ m/s². The spin component is never touched, so the policy can always re-aim. The filter sits after DOB-lite when both are enabled, so the safety arm keeps final authority, and the weights are the same frozen `bc_init_v3/model_0.pt`. The protocol is 4 environments (the env guard forbids more envs than terrain tiles), 2405 steps, `--fixed_block_prob 1.0`, seeds 42/1/7, giving 22–24 episodes per arm on the ring map and 4 on slalom.

On the ring map the cone cuts collisions by 23 % (three-seed mean 0.799 → 0.613) while blocked-goal success slightly *improves* (0.331 → 0.358). Per seed: success 0.227 → 0.348, 0.348 → 0.318, 0.417 → 0.409 against collision 0.864 → 0.565, 0.783 → 0.591, 0.750 → 0.682 (Fig. 8). Slower approaches near walls remove bounce-offs that used to end episodes, and the untouched spin component preserves the policy's ability to re-aim. In the source world, on the 256 envs × 2405-step protocol, the filter is nearly free: blocked success 0.287 → 0.274 (within per-seed noise), collision 0.642 → 0.571 (−11 %), TTG 10.95 → 11.02 s. A safety interface that taxes nominal operation is not deployable, and this one does not. Intervention rate scales with deployment stress: 46.3–51.4 % of steps (4450–4945 of 9620) on the out-of-distribution ring against 14.6 % (89,930 of 615,680) in distribution. The filter works hardest where transfer is weakest, which is the same pattern the shield shows at campaign scale (Sec. VI-G).

On the slalom map the cone does nothing. Slalom failures are side and oblique contacts while threading narrow gaps, and there the cone is a no-op: 0.2 % interventions (17 of 9620 on seed 7, zero on seeds 42 and 1), identical 0.000 success and identical collision (1.000 / 1.000 / 0.500 on both arms). A barrier that sees only the travel direction cannot see a wall the robot is *drifted sideways into*. An earlier version of the same idea was worse than a no-op and is worth recording: v0 took the minimum range over the whole ±90°

hemisphere (d_{margin} = 0.15 m, a_{dec} = 1.5 m/s²) and never fired once, 0 of 9620 interventions, because a hemisphere minimum is dominated by corridor side walls at ≈0.4 m grazing incidence. It tracked the walls the robot drives *past* and never the wall it drives *at*. An apparent improvement from v0 turned out to be a floating-point round-trip artifact ($(v \cdot x)/x \neq v$ perturbing chaotic trajectories); returning the action untouched when no clamp fires made the no-op path bit-exact, and that is what exposed the fact that v0 never clamps at all. We kept that discipline: an interface's do-nothing path must reproduce the unprotected run exactly, so that every measured difference is attributable to an actual intervention (tier 1's pass-through in Sec. IV-D publishes the policy twist itself, not an approximation of it). The slalom negative is the direct motivation for the hull-aware, persistence-backed shield: the cone demonstrates the mechanism (sensor → barrier → projection over a frozen policy), but a deployable safety layer needs a footprint model instead of a cone.

Caveats, printed next to the numbers: the custom-map arms are small (4 envs ⇒ 22–24 episodes on ring, 4 on slalom), so per-map values are seed-means and ring seed 42 is the cleanest single comparison; clearance is sampled by a 15 Hz LiDAR under 20 Hz control, i.e. one env-step stale, and the 0.30 m margin carries the hull extent and that staleness jointly rather than separately; and the filter assumes the nominal speed scale $v_{full} = R \cdot W_{nom}$, so on degraded dynamics it must be paired with DOB-lite, which restores that scale (Sec. VI-C).

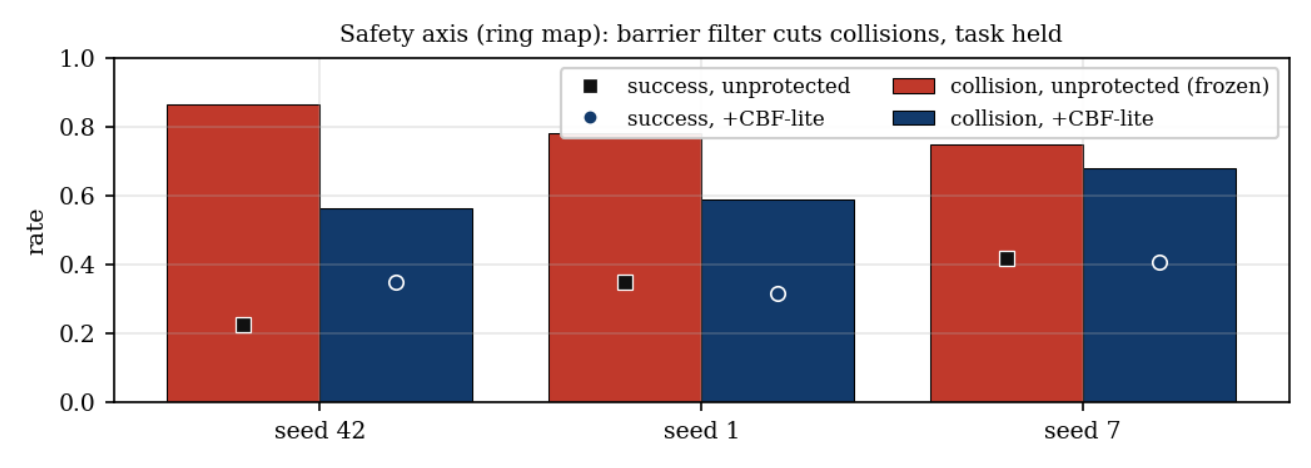


Fig. 8. Cone-barrier A/B on the ring map, per seed, frozen weights on both arms: collision rate (bars) falls under the filter on all three seeds (0.864 → 0.565, 0.783 → 0.591, 0.750 → 0.682) while blocked-goal success (markers) is held or improved (mean 0.331 → 0.358). Interventions 46.3–51.4 % of 9620 steps. On slalom the same filter is a no-op (0.2 % interventions, identical 0.000 success), the negative that motivated the hull-aware shield. *Placement: full-width (figure*), page 7. Generated by make_figures.py from logs/p4_cbf_*.log; ledger §8.*

E. Safety axis, part 2: the presentation runs and the zero-contact iteration

The five presentation maps are hand-picked and we say so: they are the demonstration set, and the systematic evidence is Secs. VI-G and VI-H. Their value here is different. They are the only runs deployed through the complete ROS 2 node stack on the second embodiment, and they were run twice with the same frozen checkpoint, the same maps and the same launch procedure, the shield enabled by one parameter. That pairing isolates the guarantee arm's contribution on identical competence.

Iteration 1 ran the competence arm alone. All five runs reached their goals, and the exact-box audit found contact in four of them (Table III, Fig. 9). The contact decomposes into three classes with different meanings. (i) *Approach grazes*: open 6 frames at ≤1.4 cm depth, mid 1, crossings 1, which are shallow, brief, and precisely the frames a coarse instrument can miss. (ii) *A sustained wedge* on the hard rung: gauntlet 917 frames, goal never reached, the policy pinned into the gap corner. (iii) *Post-success grinding* against the goal pocket, which is the bulk of every full-run count (tour 71 frames, all after the final arrival; open 453 of its 459; crossings 411 of its 412), a direct consequence of a frozen policy that has no stop-at-goal behaviour. These contacts are the expected behaviour of a policy trained with contact as a recoverable one-shot penalty and deployed without a guarantee arm. Iteration 1 measures the gap the framework predicts, and it is why every audit in this paper is split into approach and full-run columns.

Iteration 2 turned the guarantee arm on, with the same checkpoint, the same maps and the same launch procedure. All five reach their goals with *zero* hull-penetration frames over the full recorded run (0 of 1950 frames on each), with minimum hull clearance +0.131 to +0.287 m (Fig. 10); Fig. 11 plots the five executed paths on their wall sets. Recoveries drop an order of magnitude (open 20 → 1, mid 14 → 1): the shield removes stall and blip cascades at the source instead of letting the recovery layer clean up after them. The hard rung flips from failure-with-contact to success-without-contact (29.10 s, one recovery), so removing the contact cascade lets the policy's own competent steering accumulate progress. Shield activity is sparse where the policy is competent, 38 to 86 intervention ticks out of 2400, and exactly zero on one of the two tour runs (Fig. 12), which means that on that run every command the policy issued was already certified safe and the filter published it unmodified.

Table III. Presentation runs under the exact-box audit (ledger §15, report §5–§6). Approach = up to first arrival at the final goal; full = entire recorded run; min clr. = approach-phase minimum (gauntlet iteration 1 never arrives, so its full-run minimum is shown). Times are policy-clock time-to-goal. Iteration-2 penetration counts are recomputed from the kept traces with the 13-point audit hull of Sec. V-B; the in-situ raster audit logged at the time reported 458/352/1/888/71 full-run frames on open/mid/crossings/gauntlet/tour. The crossings gap is coverage (446 frames audited versus the 1950-frame trace kept, the difference being post-success grind) and the open/gauntlet gaps (1 and 29 frames) are the raster-versus-exact instrument difference. Iteration-2 rows were produced by the ROS 2-era shield (v4 on the four single-goal maps, v5 on the tour), not by the v7.5 shield of Secs. VI-G/H; see the version note below.

map	iter.	verdict	time	recov.	shield ticks	approach pen.	full pen.	min clr.
open	1	SUCCESS	29.3 s	20	—	6	459	−0.014 m
open	2	SUCCESS	26.2 s	1	67/2400	0	0	+0.182 m
mid	1	SUCCESS	26.2 s	14	—	1	352	−0.005 m
mid	2	SUCCESS	25.0 s	1	47/2400	0	0	+0.179 m
crossings	1	SUCCESS	33.5 s	2	—	1	412	−0.003 m
crossings	2	SUCCESS	24.6 s	1	38/2400	0	0	+0.287 m
gauntlet	1	FAIL (wedge)	—	25	—	917	917	−0.062 m
gauntlet	2	SUCCESS	29.1 s	1	86/2400	0	0	+0.158 m
tour	1	SUCCESS 3/3	46.1 s	3	—	0	71	+0.331 m
tour	2a	SUCCESS 3/3	49.9 s	0	0/2400	0	0	+0.237 m
tour	2b	SUCCESS 3/3	57.2 s	0	45/2400 (20 esc.)	0	0	+0.131 m

Two generations of the same mathematical filter appear in this paper, and we label them everywhere instead of pooling them quietly. The presentation runs of Table III were produced by the ROS 2-era shield (v4 on the four single-goal maps, v5 on the tour runs; Stack B of Table I): two tiers, a 17-point hull, an obstacle model built only from persisted LiDAR hits, and a stop command treated as axiomatically admissible. The campaign (Sec. VI-G) and the held-out endgame (Sec. VI-H) were produced by v7.5: three tiers, a 51-point hull, the exact-box ESDF (Stack A) or its union with the persisted raster (Stack B), stop certified by evaluation rather than by axiom, and the never-publish-stop-while-a-certified-mover-exists rule. Both appear because the second generation exists as a result of stress-testing the first: the same 17-point/raster instrument, run on the tighter 10 m procedural pool, penetrated twice, at −0.0405 m and −0.0564 m (Sec. IV-D1), with a hull corner reaching between sample points into a 0.12 m-thick box that the 0.08 m raster had quantised away. The presentation set's zero contact is therefore a true measurement of a weaker instrument on geometry that did not stress it, and we do not extend that claim beyond the maps measured. Every guarantee statement in this paper is relative to the obstacle model *and* the hull model of the version that produced it. Scaling a safety shield from a five-map proof of concept to a 708-episode campaign cost, in order, a sector veto that protected the sensor instead of the hull, a spawn deadlock, a post-success grind, a corner-pocket livelock, two hull penetrations, and an absorbing freeze. Each one was measured, root-caused, fixed and kept.

The chronology behind those versions follows, and every artifact is kept. *v1 (sector veto)*: twelve angular sectors, per-sector minimum range, unsafe sectors vetoed. The robot reached the goal and the audit still flagged penetrations, because sector minima protect the sensor origin rather than a hull with corners. *v2 (hull-aware, margin 0.22 m, no persistence)*: deadlocked at spawn; every candidate including in-place rotation was judged unsafe, so it published (0, 0) forever, with 0.08 m of total motion and 2400/2400 shield ticks. Diagnosis: the spawn corner's true hull clearance was +0.31 m, less than margin plus dilation, and the measured LiDAR bias was ±0.062 m, which is cell quantisation rather than calibration error. Two lessons follow: size the margin against the real clearance distribution, and in a static world remember obstacles instead of re-sensing them. *v3 (margin 0.10 m, reverse candidates, per-beam world-frame persistence)*: reached the goal on demo_open with 206 flagged frames, *all after success*, because the frozen policy ground the goal pocket for the remaining 97 s (approach phase: 0 penetrations, +0.17 m). The fix was behavioural, park-after-success, not a relaxation of the barrier: final audit 0 penetration frames, exact and raster. *v4 (presentation set)*: cleared four maps, livelocked on the fifth. *v5 (two-tier)*: tier 2 engages only when tier 1 is rotation-locked, and re-checks candidates against the raw wall grid (cells containing actually-sensed wall points) at $|v| \le 0.20$ m/s, where raw-grid safety, meaning the hull never entering a sensed-wall cell, is the hard no-touch bar, so zero contact remains guaranteed by construction. On maps that never rotation-lock the escape tier is provably inert, and it was verified offline in both directions: a fresh buffer passes tier 1 with no intervention, and a worst-case union buffer makes the robot stop safely rather than move unsafely.

The first shielded tour run (v4) livelocked at tour goal 3: in the tight corner pocket, grazing hits plus dilation left no forward candidate admissible in tier 1, and the robot rotated in place 9 mm short of the success radius, indefinitely. Exact geometry showed the pocket was open: nearest walls 0.455/0.465 m, closest approach 0.309 m against a 0.30 m success radius. The mechanism was sensor noise rather than geometry: grazing LiDAR hits scatter 3–20 cm off true wall surfaces, and the disk dilation ($K = \text{ceil}(0.10/0.08) = 2$ cells ≈ 0.16 m) pushed the comfort envelope past the ≈0.18 m clearance the hull actually keeps. Tier 2 resolves it without weakening the guarantee, and the two independent v5 tour runs show both paths through the same pocket: one threaded it with zero shield interventions (49.9 s, minimum clearance +0.237 m), the other rotation-locked and crept out through the escape tier (20 escape ticks of 45 total, 57.2 s, minimum +0.131 m). Both are 3/3 goals with zero contact. Run-to-run ROS/sim timing selects the path, while the guaranteed quantity,

the contact outcome, held in both. The claim is scoped to the contact outcome and not to the trajectory, and we report the run-to-run spread rather than the favourable member. The livelock trace is kept as an artifact.

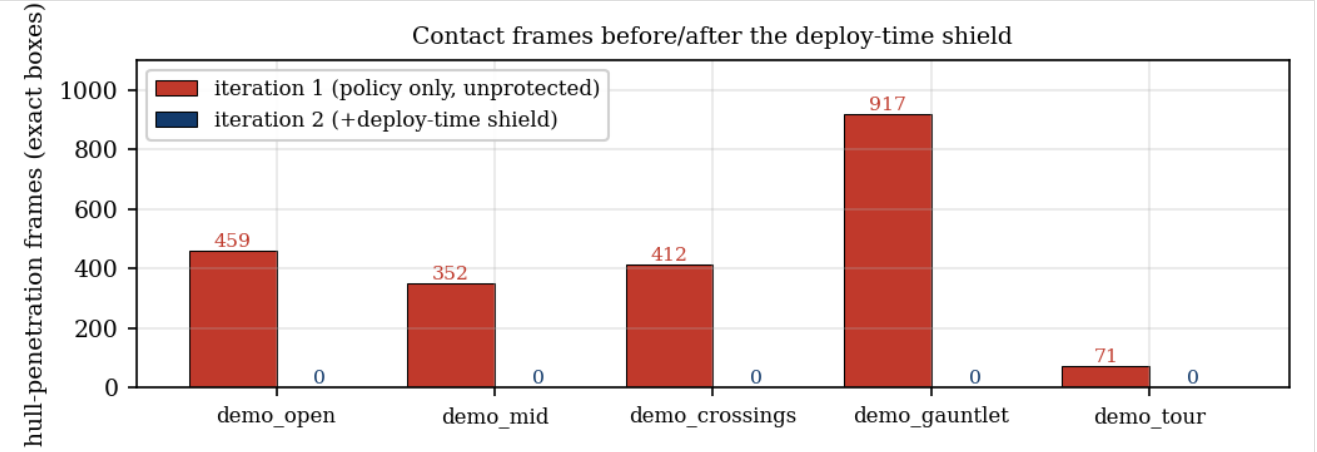


Fig. 9. Hull-penetration frames under exact wall geometry, full recorded runs, recomputed from the kept traces with the 13-point audit hull: iteration 1 [459, 352, 412, 917, 71] versus iteration 2 [0, 0, 0, 0, 0] on open/mid/crossings/gauntlet/tour. Iteration-1 full-run counts are dominated by post-success grind; Table III splits approach from full. *Placement: full-width (figure*), page 8. Generated by make_figures.py:fig_penetration from videos/custommap_demo/ and videos/ custommap_shield/ traces.*

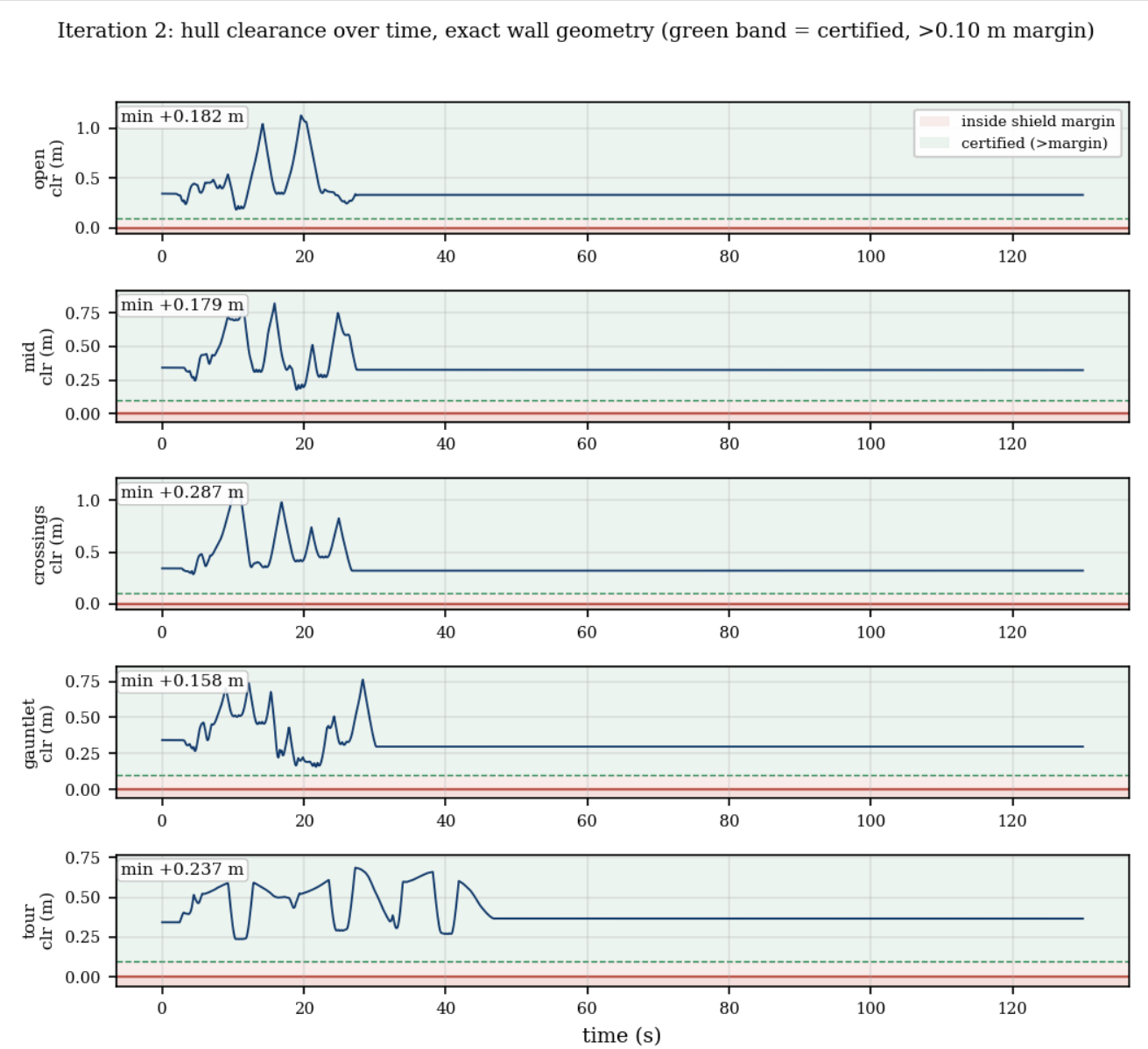


Fig. 10. Hull clearance over time for the five shielded runs under exact wall geometry, recomputed from traces (navy = shielded arm). The curve never crosses zero; annotated minima (+0.182, +0.179, +0.287, +0.158, +0.237 m) match Table III. The pale-red band is the shield's 0 → 0.10 m margin (inside it a command is filterable, below zero it is a penetration), the thick red line at zero is the exact wall surface, and the pale-green band above the margin is certified clearance. The tour panel spans two leg transitions; the confirmation tour run (not plotted) has minimum +0.131 m and is the run whose escape-tier ticks appear in Fig. 12. *Placement: full-width (figure*), page 8. Generated by make_figures.py:fig_clearance.*

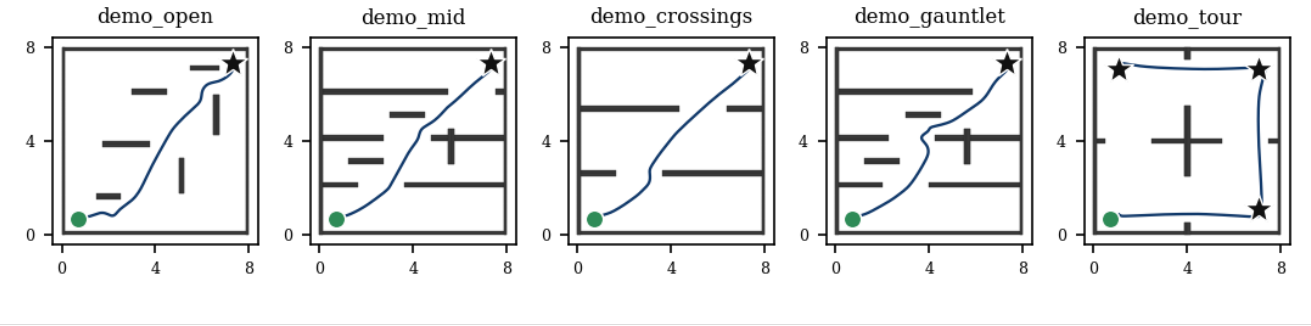


Fig. 11. Shielded trajectories on the five presentation maps (walls grey, spawn green dot, goal black star, path navy = frozen policy plus shield). The gauntlet S-detour and the three-leg tour are composed by the frozen policy plus interface: no weights change between maps, only the geodesic field rebuilt from each map's JSON. *Placement: full-width (figure*), page 8. Generated by make_figures.py:fig_traj from iteration-2 traces and map JSONs.*

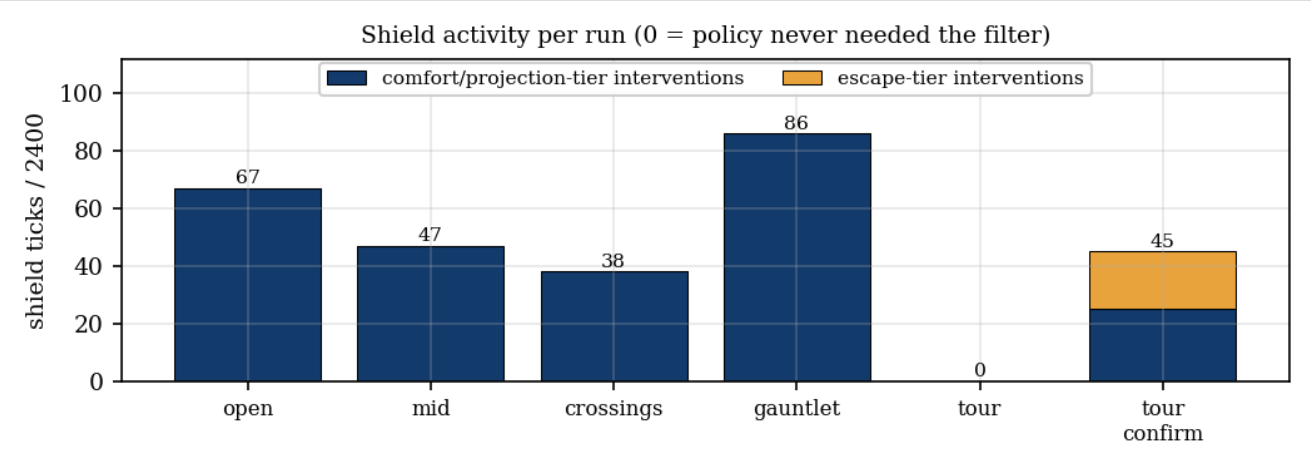


Fig. 12. Shield interventions per run, parsed from the run logs and split by tier (navy = comfort/projection, amber = escape). Zero interventions on tour run 2a means every policy command was certified as issued; the escape tier fires only in the run that rotation-locked in the goal-3 pocket (20 of its 45 ticks). The four single-goal runs are v4 logs, the two tour runs v5. *Placement: full-width (figure*), page 8. Generated by make_figures.py:fig_interventions from logs/shield_policy_*.log.*

F. Task and deployment axes

Task structure transfers through the interface. The ordered three-goal tour is solved by the frozen policy with per-leg field rebuilds: the node reads an ordered `goals` list from the same map JSON, advances on arrival (< 0.30 m) and rebuilds the geodesic field per leg, taking 46.05 s for all three legs in iteration 1 (leg arrivals at 14.40 / 29.00 / 46.05 s, 3 recoveries) and 49.9 and 57.2 s shielded. Sequencing is an interface arm, so one learned behaviour set composes into many task instantiations with zero weight changes. Its negative counterpart is informative: on `demo_serial` (true 2 × 2 rooms, 1.0 m doors) the same frozen policy stalls at the leg-1 door, 0/3 goals, 3.97 m short, 20 recoveries, which is the identical corner-wedge seam that fails the ladder's hard rung (Sec. VI-B) on an independent map family. The boundary is a property of the transferred behaviour set, not of one map.

Deployment transfers through the message boundary. The same node stack drives a pioneer3dx plant with wheel radius 0.09 m against the trained 0.050353 m, track 0.32 m against 0.324 m, different mass, casters and a velocity servo (KD = 20), through the identical `geometry_msgs/Twist` on `/cmd_vel`. Nothing above that boundary knows which robot is below it: linear speed is preserved exactly by the chain and angular speed within 1.25 % (the ≈1.2 % track mismatch is absorbed by the servo). What changes at deploy time is small and enumerable: the kinematic constants in the action → twist conversion, the footprint inflation used to build the field (0.20 m in Lab, 0.35 m for the larger hull), and the stall-proxy calibration. What does not change is the 1208-dimensional observation contract, the weights, the field construction and the recovery triggers. Evidence on the mismatched plant: the run drives spawn (0, 0) → goal (−7, 4), an 8.07 m reach, in 30.85 s with zero retraining. The companion result on the same plant is that the detour-conflict goal (6, 8) was NOT REACHED, with 8 recoveries. This is deploy-time embodiment transfer under a velocity-level interface; it is *not* cross-embodiment *learning*, and it is not the latent-alignment mechanism proposed in the framework draft, which remains unexercised anywhere in this paper and is queued as future work (Sec. VIII). Table IV consolidates every transfer axis examined above under a single frozen checkpoint.

Table IV. Transfer ledger: every axis examined in this paper under one frozen checkpoint (`bc_init_v3/model_0.pt`), condensed from ledger §3, §6–§15 and §17. "Mechanism" is the interface component that carries the axis; the weights never change in any row.

axis	mechanism	result
geometry (zero-shot)	none	rail yard 0.190 blocked succ. (−34 % vs source 0.287)
geometry + task	GeoField from the shared map JSON	user maps zero-shot; ladder easy/mid 29.30/26.15 s; 3-goal tour composed in 46.05 s
robustness	spawn jitter ±0.15 m / ±15°	4/4 success, 20.9–33.5 s
interface necessity	waypoint-blinding ablation	4/4 → 0/1, pins 5.74 m short
dynamics	DOB-lite observer	degraded 0.028 → 0.363; within 3.2 % of an oracle told the factor
dynamics model	frozen Koopman lift + refit	wheel channels 0.88 → 0.57 → 0.99 (refit); lift barely beats affine
safety (training world)	cone barrier (CBF-lite)	ring collisions −23 %, success 0.331 → 0.358; slalom no-op
safety (ROS 2 deployment)	two-tier shield, v4/v5 (Stack B)	0 contact frames on 5 maps + 2 tour runs, exact-box audit
adaptation budget	retrain-from-scratch curve	zero-shot ≡ ≈238 k target-world demos
embodiment	velocity-level interface	pioneer3dx plant, 8.07 m reach, 30.85 s, zero retraining
safety at scale	three-tier shield v7.5 (Sec. VI-G)	56 maps × 3 arms, 708 episodes: thesis arm 0.979 succ., 0.000 collision
held-out geometry	same shield, no map-specific tuning	user-drawn maze: thesis 1.000/0.000 vs unprotected 1.000/0.900; trained 0.000/0.000

G. The systematic campaign: 56 procedural maps, three arms

The presentation runs are hand-picked; the campaign is not. We generated 56 maps from 7 procedural families (corridor, gates, loop, fortress, competition, warehouse, highway) × 4 difficulty bins × 2 variants, and ran every map under three arms at a fixed seed with 4 spawn-yaw repeats (10 on the two locked presentation maps): the frozen trained policy under the shield, our in-house A*+DWA planner *unprotected*, and the same planner *wrapped in the shield*, which is the thesis arm, where a competent but uncertified controller receives its guarantees from the interface alone. The campaign is 236 episodes per arm, 708 total, shield v7.5 (Stack A) on every shielded run, 4.5 h aggregate wall clock, 0 timeouts, 0 non-zero return codes, 0 incomplete episode captures.

Table V. Three-arm campaign, episode-pooled over 56 maps (236 episodes/arm; ledger §17.20). Collision = simulator contact force > 5 N (the physical instrument of Sec. V-B); TTG = mean time-to-goal over successful episodes; interventions as a fraction of that arm's 452,404 env-steps.

arm	success	collision	TTG (s)	shield interventions
trained + shield	0.644	0.004 (1/236)	47.5	46.6 % of ticks
A*+DWA (unprotected)	0.941	0.534	52.7	—
A*+DWA + shield (thesis arm)	**0.979**	**0.000 (0/236)**	54.8	14.5 % (65,457: escape 12,684, retreat 5,327, pin 142)

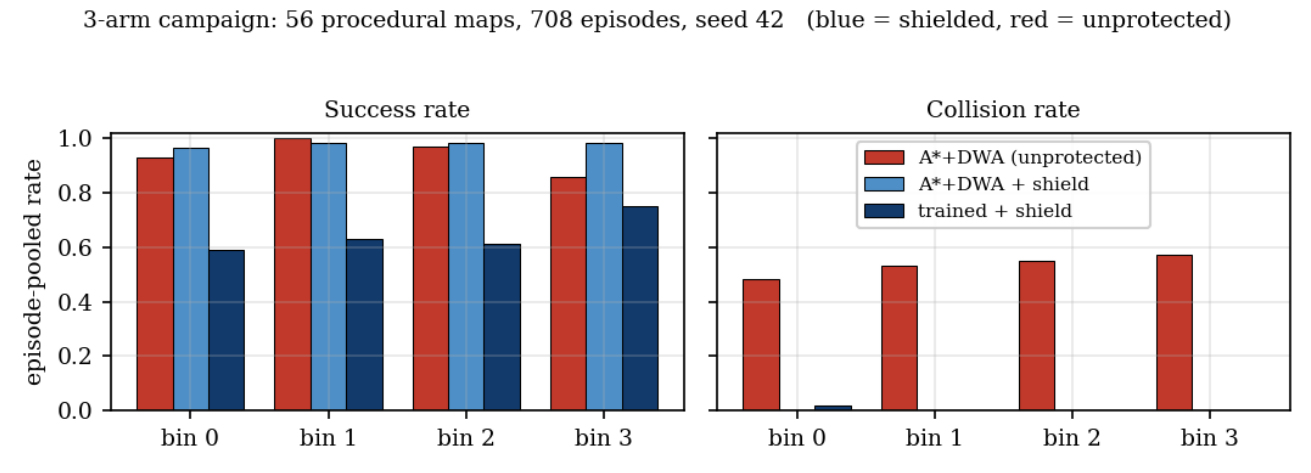


Fig. 13. Campaign per difficulty bin, episode-pooled over 56 maps, in the paper-wide colour grammar (red = unprotected, steel blue = classical planner + shield, navy = trained policy + shield). Unprotected collisions grow with difficulty (bin 0 0.48 → bin 3 0.57) while the thesis arm is flat at 0.000 in every bin and even gains success (0.941 → 0.979 overall, bin-wise 0.96/0.98/0.98/0.98: DWA stalls become escape-tier recoveries instead of timeouts). The trained arm's collisions are equally flat but its success is family-structured (text). *Placement: full-width (figure*), page 9. Generated by make_figures.py:fig_campaign from campaign/aggregate.json.*

Three results carry the campaign. (i) The guarantee held at scale. The thesis arm scored zero collisions on all 56 maps, 0 of 236 episodes, and the off-model audit (`shield_oob`, ticks where the robot left the barrier's map-rectangle domain) counted zero over 452,404 env-steps for each shielded arm. The unprotected planner contacted walls in 53.4 % of episodes, concentrated where geometry is tightest (fortress 0.95, warehouse 0.74, corridor 0.72) and nearly absent on the wide highway family (0.03): the failure class is corner-cutting in narrow space, which is what a hull-aware barrier certifies against. Its own diagnostics confirm the mechanism: 23,062 plans, 22,826 replans, 0 stuck episodes, and `dwa_reject` = 0 over the whole campaign, so DWA's 0.03 m point-model margin never once rejected a candidate and nothing in the planner itself opposed those contacts. (ii) The shield is cheap and difficulty-flat. It costs +2.1 s TTG (+4 %), intervenes on 14.5 % of ticks, and its collision rate does not move across difficulty bins (Fig. 13). It also *adds* competence: success rises 0.941 → 0.979 because stall and livelock episodes become escape-tier recoveries instead of timeouts. (iii) The learned arm has a negative. The trained policy's competence is family-structured: 0.94 on competition and highway, 0.84 on loop and warehouse, 0.61 on fortress, 0.31 on gates, 0.000 on corridors (31 of 32 corridor episodes timed out, plus the one scrape). 83 of its 84 failures are timeouts, not contacts, so the shield holds safety exactly where the policy fails the task. That single shielded contact in 708 episodes is the marginal 5.6 N tier-3 scrape on `f1_b0_m01` (trained arm, env 2, against a 5 N threshold), root-caused to the uncertified retreat tier (Sec. IV-D3). We report it as the campaign's one breach rather than rounding it away, and the tier that produced it is the only uncertified line of code in the shield.

Two further numbers are printed here because a mean hides them. On the two locked presentation maps, run with 10 spawn-yaw seeds each, the thesis arm scores 1.000 success / 0.000 collision at 65.5 s (`f4_b1_m00`) and 1.000 / 0.000 at 58.7 s (`f6_b2_m00`), against 1.000 / 1.000 unprotected at 59.1 and 54.5 s: on the maps we use for demonstrations, the unprotected planner collides in every single episode and the same planner under the shield collides in none. The trained arm on those two maps is 0.600 / 0.000 and 1.000 / 0.000, so safety holds again where competence is uneven.

Two stops are documented. The campaign as tabled is the third launch. The first (two arms, 112 runs planned) was stopped after 8 runs when the shield arm returned 0/4 on three consecutive corridor maps while the unprotected planner solved all of them: rather than spend four hours on a measurement we could not interpret, we diagnosed it. A bare-policy trace presses into a wall for the final 76 s of the horizon (0.16 m of creep); a shielded trace is correctly rerouted around that wall by the shield and then wanders off-route and freezes

spinning in an *open* pocket from t = 77.5 s to timeout while the waypoint observation still points back along the connected route. That is an out-of-distribution state for a policy trained on open terrains, since it has no behaviour for “the waypoint says go back the way you came”, and every shielded run in that batch recorded zero collisions: the guarantee held, the competence failed. The third arm exists because of that stop. The second launch was stopped after 3 runs when `f1_b0_m01` logged 1952 out-of-domain ticks on the thesis arm; cell-by-cell inspection exonerated the map (its east border is two full-height segments forming a sealed T-junction) and the cause was a PhysX spawn pop, fixed with a 5 mm spawn lift. The same window produced two reproducibility fixes we kept: the generator's per-cell boxes were merged 13,395 → 768 (−94 %) with a cell-for-cell geometry-preservation proof, and a family that was silently non-reproducible across processes (set-iteration order over strings, i.e. PYTHONHASHSEED) was made deterministic, verified by identical geometry hashes. Pre-fix contaminated results remain archived rather than deleted.

One counterexample qualifies the guarantee and is treated in Sec. VII-C: a single-environment replay of `f6_b2_m00` under the thesis arm *does* record a collision, while the 10-environment protocol above records 0.000 on the same map. Both are correct measurements of different protocols, and the frame-exact audit of that replay shows the certified hull never penetrating (minimum +0.070 m over 2711 frames), so the contact is drive hardware outside the hull box.

H. The held-out endgame: a map drawn by the user, unseen in training and in the campaign

The campaign's 56 maps are produced by our own procedural families, so a sharper test of “the guarantee comes from the interface” is a map nobody engineered. A user drew an arbitrary serpentine maze on a blank 800 × 800 canvas with no straight shot from start to goal, and we ran it with no retraining, no map-specific tuning and the *identical* locked protocol: 10 spawn-yaw seeds folded into `num_envs`, seed 42, horizon min(34.38/0.25 + 10, 190) = 147.52 s, 2971 steps, field iterations auto-raised to 177 for convergence on the long path. The drawing rasterises to a 15.0 × 15.0 m world (0.125 m cells, 884 wall boxes, 0.5 m wall height), spawn [7.812, 2.312], goal [7.812, 11.938], geodesic path 34.38 m, narrowest corridor ≈1.11 m with clearance ≥0.50 m along the whole path (Fig. 14). It appears in neither the training distribution nor the campaign.

We report the one judgment call this map required. At the native 8 m tile the narrowest corridor was ≈0.75 m, whose clearance-restricted bottleneck is ≈0.37 m: the 0.33 m hull half-diagonal barely fits, but the inflated A* planner (which needs ≈0.53 m) cannot plan through it, so the run would have measured the planner's inflation parameter rather than the framework. The naive metric we first used, minimum clearance along the BFS shortest path at 0.125 m, was misleading, because that path hugs walls; the correct instrument is a clearance-restricted BFS over {free ∧ D ≥ MINC} with D the Euclidean distance transform. Rescaling the physical tile size to 15 m changes corridor metres without changing the cell count or the drawing, and at MINC = 0.50 m the map is navigable everywhere. The 884 boxes merge into a 113,248-vertex / 509,304-face terrain mesh in 3.2 s; because the barrier's field queries are O(1) on a precomputed 0.025 m lattice, per-step shield cost is unchanged from the 10-box campaign maps. All three arms ran sequentially, one simulator at a time, in ≈10 minutes total (165 / 202 / 224 s), with no crash and no out-of-memory event.

Table VI. Held-out endgame: one user-drawn map, unseen in training and in the campaign, identical locked protocol (10 spawn-yaw seeds, seed 42, 147.52 s horizon; ledger §17.22). Collision = simulator contact force > 5 N; TTG = mean time-to-goal over successful episodes; ticks out of 29,710 per arm.

arm	success	collision	TTG (s)	oob	shield ticks	planner
trained + shield	0.000 (0/10, all timeout)	0.000 (0/10)	—	0	22,549 = 75.9 %	—
A*+DWA (unprotected)	1.000 (10/10)	0.900 (9/10)	103.3	—	—	plan 1500, replan 1490, dwa_reject 0, stuck 0
A*+DWA + shield (thesis arm)	**1.000 (10/10)**	**0.000 (0/10)**	105.7	0	5,711 = 19.2 %	bit-identical to the unprotected twin

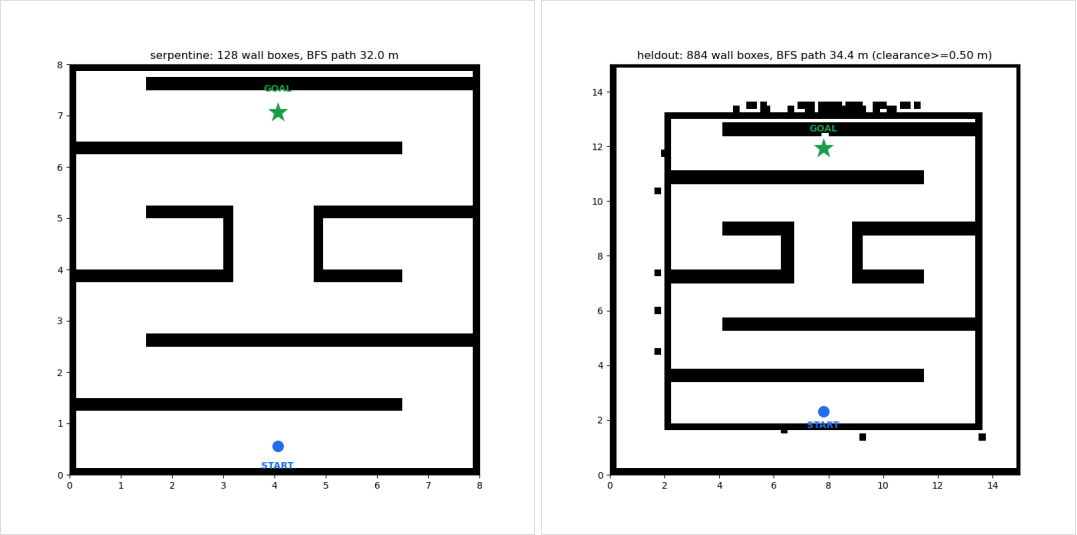


Fig. 14. The held-out endgame map: the user's original drawing (left, 800 × 800, spawn bottom-centre, goal top-centre) and the BFS-validated preview of the generated world spec (right, 15.0 × 15.0 m, 884 wall boxes, 34.38 m geodesic, narrowest corridor ≈1.11 m). The same JSON feeds the simulator colliders, the planner's costmap and the shield's exact-box distance field. *Placement: full-width (figure*), page 9. Source images: openmover_ws/isaac/lab/maps/healdout.png (filename as committed) and heldout.preview.png.*

The three arms separate cleanly on this single unseen map, exactly along the framework's axis. (i) The learned policy does not transfer to this geometry, and the shield cannot manufacture competence. The frozen checkpoint reaches the goal in 0/10 episodes, and every seed runs out the 147.52 s horizon (per-env time-to-goal 147.5 s, reached = false throughout). This is competence failure, not truncation: both planner arms finish in ≈104 s with ≈30 % of the budget to spare, while the trained arm consumes all of it and arrives nowhere, which reproduces the corridor/gates gap of Sec. VI-G on a map no procedure generated. The shield works far harder here than in distribution (75.9 % of ticks versus 46.6 % in Table V; escape 1546, retreat 1335, pin 252, and no livelock) yet still holds safety exactly where the policy fails: 0/10 collisions, zero out-of-domain ticks. (ii) The classical planner transfers but is not safe by itself. A*+DWA plans on the occupancy grid of *any* map, so its competence is map-agnostic: 10/10 success. Unprotected it contacts walls in 9 of 10 episodes (only env 1 is clean), and its own DWA margin never once rejected a candidate (`dwa_reject` = 0 with a 0.03 m margin): the point model deemed every command safe while the hull and the drive hardware clipped the ≈1.11 m corridor walls. (iii) Wrapped in the shield, it is competent *and* guaranteed. The thesis arm runs the identical planner (plan 1500, replan 1490, dwa_reject 0, stuck 0, bit-identical to its unprotected twin) and converts 9/10 collisions into 0/10, zero out-of-domain ticks, for +2.4 s (+2.3 %) TTG. The exact-box barrier intervenes on only 19.2 % of ticks, because the planner already drives mostly safely and the filter's job reduces to vetoing the corner-cuts DWA's margin waves through. This extends the thesis arm's campaign guarantee from 0/236 to 0/246 episodes with no map-specific adjustment: on a map drawn by a person and never seen in training, the guarantee still comes from the interface, not from the policy.

Both contact instruments were run on this map. Auditing the clipped episode-0 traces under exact box geometry gives minimum hull clearance +0.1546 m with 0 of 2082 penetration frames for the thesis arm, +0.0703 m with 0 of 2135 for the *unprotected* planner, and +0.1194 m with 0 of 2950 for the trained arm. The geometric instrument therefore finds no hull penetration on this map under *any* arm, while the physical instrument records 9 collisions for the unprotected planner. Both are correct, and the gap between them is the substance of Sec. VII-B: the contact is drive wheels and casters, hardware that sits outside the certified 0.44 × 0.28 m hull box. This also bounds our own claim: the shield certifies a hull model, and on this map the unprotected arm's collisions were never hull penetrations, so the thesis arm's 0/10 is a statement about contact forces and not a strictly stronger geometric margin than its twin's. Fig. 15 renders this single episode frame by frame, one column per arm: what each arm drove (top) and how close the certified hull came to the walls while it did so (bottom).

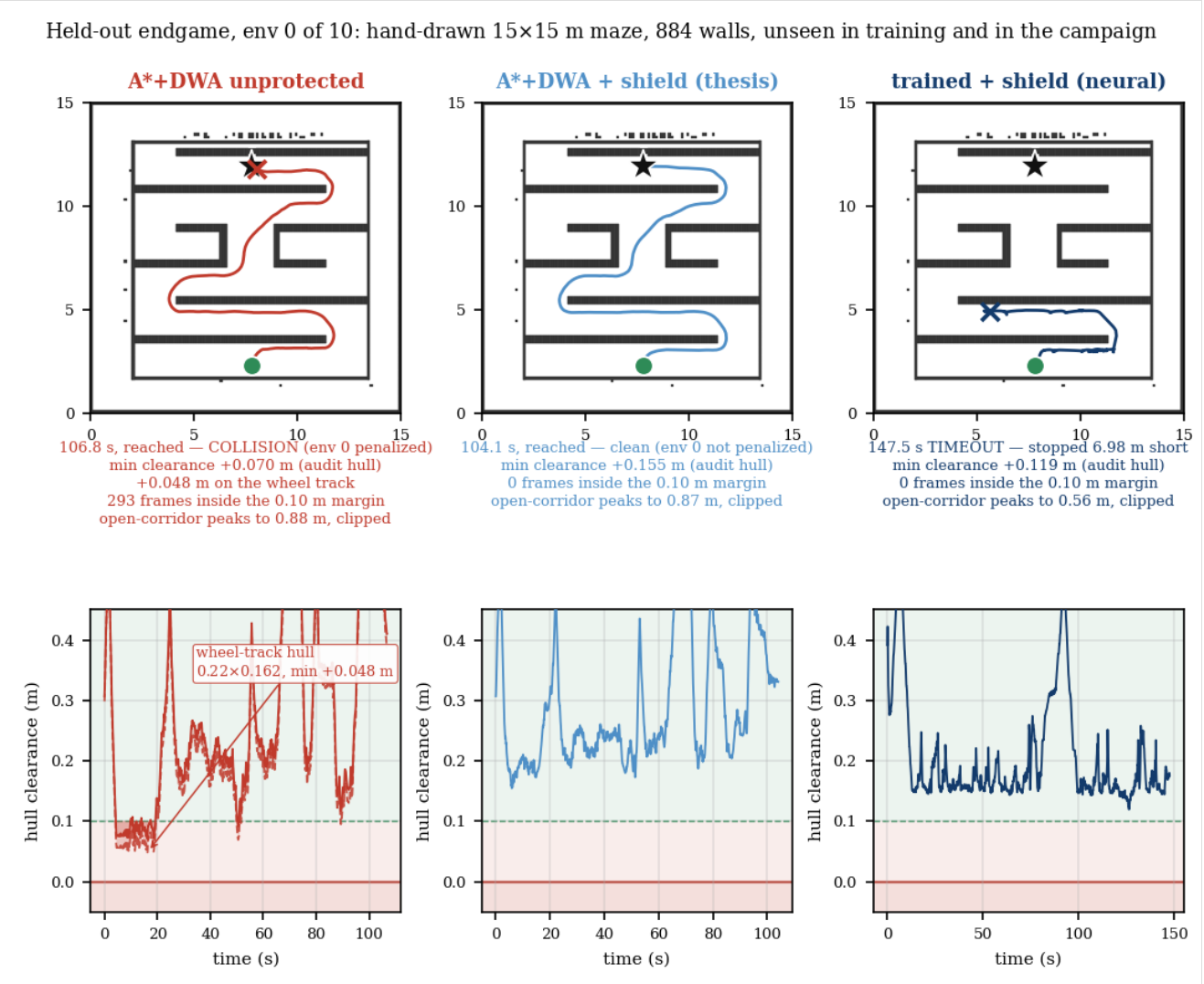


Fig. 15. The held-out endgame, episode 0 of 10, at 20 Hz: executed trajectories over the 884-box wall set (top row; green dot = spawn, black star = goal, cross = horizon-end position of the timed-out arm) and hull-clearance time series under the 13-point audit hull (bottom row; pale-red band = inside the 0.10 m shield margin, thick red line at zero = exact wall surface, pale-green band = certified clearance). **Left, red, A*+DWA unprotected:** reached in 106.8 s but the env-0 contact latch fired (one of its 9/10 collided episodes); minimum clearance +0.070 m on the audit hull and +0.048 m on the wheel-track hull (dashed, 0.22 × 0.162 m), with 293 frames spent inside the margin. **Centre, steel, A*+DWA + shield (thesis arm):** reached in 104.1 s, env-0 clean (one of its 10/10 contact-free episodes); minimum +0.155 m and zero frames inside the margin. **Right, navy, trained + shield:** consumed the full 147.5 s horizon and stopped 6.98 m short of the goal, a competence failure and not a safety failure; minimum +0.119 m and zero frames inside the margin. Open-corridor peaks (0.88 / 0.87 / 0.56 m) are clipped at 0.45 m for readability. Under the exact-box audit all three arms record zero hull-penetration frames on this episode; the unprotected arm's contact is wheel and caster force outside the modelled hull (Sec. VII-B). Colours follow the paper-wide grammar: red = unprotected, blue family = shielded. *Placement: full-width (figure*), half page, page 10. Generated by make_figures.py:fig_heldout from campaign/heldout/videos_present/videos/heldout_*_clip.npz and campaign/heldout/results/<arm>/heldout.json.*

VII. DISCUSSION

A. What the results do and do not prove

Taken together the results support one sentence: *a learned policy provides competence that can cross worlds, tasks, dynamics and embodiments with frozen weights, and a mathematical interface provides the guarantees and task grounding that the learned component cannot, and the two are separable enough that each can be measured, and replaced, independently.* Each clause is measured: transfer by the ladder, the seed repeats and the budget curve; the competence-carrier by the waypoint-blinding ablation (4/4 → 0/1, pinned 5.74 m short); guarantees by the exact-box audit and the >5 N contact latch across 708 campaign episodes and 10 held-out episodes; and the boundary between the two arms by the shield's inability to aim at anything and the policy's inability to stay contact-free alone.

The separability clause produced the central result, and it cuts against the framing we started with. Because the guarantee arm is arm-agnostic, we could wrap a competent classical planner in the same shield and measure both arms side by side. On the campaign's 56 maps and on the held-out drawing, the shield-wrapped planner is the only arm that is both competent and contact-free (0.979 / 0.000 and 1.000 / 0.000); its unprotected twin is competent and unsafe (0.941 / 0.534 and 1.000 / 0.900); the learned policy is the only arm that crosses dynamics and embodiment with frozen weights, and on geometry far outside its training distribution its competence runs out (0.000 on corridors, 0.000 on the held-out map) while its guarantee does not (0/236 and 0/10 contacts). We report this as the benchmark's central finding: guarantees transferred perfectly and arm-agnostically, while competence transferred along some axes and not others. A paper that claimed the learned policy wins on every axis would be claiming something our own data contradicts.

Against the objection that classical planners already solve contact-free navigation on known maps, the campaign answers with a measurement: they do not, not by themselves. Our own competent A*+DWA planner contacts walls in 53.4 % of its 236 campaign episodes and 9 of 10 held-out episodes, with its internal 0.03 m margin rejecting *nothing* in either set. Wrapped in the shield it is contact-free in all of them and *more* successful (0.979 versus 0.941), because stalls become escape-tier recoveries instead of timeouts. The shield *is* a classical DWA-family layer, deliberately: the claim was never that a network dodges walls better than DWA. What a hand-written planner does not do is transfer. Retarget it to a new world, degraded dynamics or a new embodiment and it is rewritten and retuned; the single frozen checkpoint crossed all four axes through interfaces, and the same shield wrapped both controllers without a line of arm-specific code. Conversely the shield carries no task objective: with no policy and no planner there is nothing to certify and nothing to aim at. Neither arm alone does the job. The composition does, and the composition is what we offer, not either component by itself.

What the results do not prove: that a learned policy is the best competence source for this task (on our own benchmark a planner beats it); that anything here transfers to hardware (every number is simulation, with a ROS 2 boundary and a second simulated embodiment as the closest proxies); that the no-contact property is unconditional (it is relative to four models, enumerated next); or that the guarantee arm improves competence in general (it improved the planner's success by 3.8 points and could not move the trained arm's corridor failures at all).

B. Limitations and threats to validity

- Simulation only. No physical-robot validation. The ROS 2 rung, the persisted-LiDAR obstacle model, the 15 Hz sensor under 20 Hz control and the second simulated embodiment are hardware-shaped proxies, not hardware.
- The guarantee is relative to a hull model. The barrier certifies a 0.44 × 0.28 m box sampled at 51 points. The drive wheels sit at `y = ±0.162` m, 2.2 cm outside that box, and the casters outside it too. In the single-environment counterexample below, and in the held-out unprotected arm's 9 collisions, the certified hull never penetrates (minimum +0.070 m over 2711 frames; +0.0703 m with 0 of 2135 frames on the held-out map) while the simulator still latches contact on the wheels. Widening the certified hull to the wheel track is the direct fix; it is queued, not claimed, and until it lands “zero contact” in this paper means zero contact of the certified hull plus zero >5 N latch where reported.
- The guarantee is relative to the evaluation protocol. The thesis arm's 0/236 campaign collisions hold under the 10-environment protocol. A `num_envs=1` replay of the same map and seed (`f6_b2_m00`) livelocks into a wall corner and registers one >5 N contact at t ≈ 135 s before timing out; the 10-environment rerun is bit-identical to the campaign (shield 9766, hysteresis 6227, escape 2102, retreat 810, pin 27). Two correct measurements of two different protocols, and the frame-exact audit attributes the contact to the hull-model gap above rather than to a barrier failure. We keep both because a guarantee that only holds under one batching configuration is a fact a deployer needs (ledger §17.21).
- The certificate assumes the chassis tracks the published twist. The campaign's only shielded contact is exactly a violation of that assumption: at t = 88.60 s on `f1_b0_m01` the trained arm's tier-3 retreat chattered at 20 Hz between two near-tied candidates, publishing (v −0.100, w +0.300) and (v +0.100, w −0.300) on alternating ticks; every published command had *positive* predicted hull clearance (+0.047 to +0.050 m along the whole arc), and the contact (5.6 N against a 5 N threshold, in a dead-end corner) came from wheel slip and chassis lunge during direction reversals every 50 ms, which is motion outside the certified kinematic arc. The distance field did not fail; the plant did. This is why Stack A filters unrealizable candidates before certification (Sec. IV-D2), and it bounds the claim for any real chassis with slower tracking than a simulated one.
- Tier 3 is uncertified, and our fix for it made things worse. The retreat tier fires on 5327 of the thesis arm's 452,404 steps (1.2 %, a subset of the 12,684 escape-tier firings) and carries no clearance certificate; it produced that one scrape. We tried the obvious repair, the same +0.05 continuity bonus the certified tiers use, and reverted it: on a four-run validation matrix it removed the trained arm's scrape and created two new ones, *including one on the thesis arm*, failing the campaign gate. The mechanism is that in tier 3 nothing is certified and clearance gains are small (≈0.05–0.15), so a continuity bonus becomes a lock-in that self-reinforces a degrading retreat. The 20 Hz dither is net-stationary and, empirically, safer than committed uncertified motion. Pre-v7.5 the same arm scored 0.000 collisions only because the shield froze, which was accidentally safe, and v7.5 traded that hidden safety for liveness at a measurable cost of one marginal scrape in 708 episodes.
- The audit instrument is coarser than the shield. Clearance audits sample the hull at 13 points, against the shield's 51, with 0.22 m gaps along the long edges; audited clearances are therefore upper bounds on true hull clearance, and audited penetration counts are lower bounds. An earlier draft of this work described the audit hull as 17 points; the script has 13 (Sec. V-B).
- Observation-relative evidence on static maps. Stack B certifies against everything the LiDARs have ever seen and nothing else: obstacles below sensor height from the start, or occluded inside the 0.22 m near clip before first sight, are outside its evidence, and margin plus low speed bound but do not eliminate that risk. The unmapped-hit filter tolerates 0.05 m of disagreement with the exact field so that a moved crate survives in the raster, which is also what makes a stale obstacle permanent. Dynamic obstacles need bounded-staleness decay, which is not implemented anywhere in this paper.
- Two shield generations, labelled not pooled. Table III was produced by the two-tier, 17-point, sensed-raster shield (v4/v5); Tables V and VI by the three-tier, 51-point v7.5 shield. The older instrument penetrated twice on tighter procedural geometry (−0.0405 and −0.0564 m), so its zero-contact result on five unstressing maps is a true measurement with a narrow scope. Every guarantee statement here is relative to the obstacle model and hull model of the version that produced it (Sec. VI-E).

- The task interface is given the map. The geodesic field is built from the same JSON that bakes the simulator's colliders, and Stack A's barrier certifies against that exact geometry. Waypoint provisioning and the strongest guarantee both assume a known static map; on hardware that role would be filled by a mapping or SLAM stack whose error we did not test. Stack B is the intermediate case: its guarantee is relative to sensed evidence only.
- Fixed-seed statistics and hand-picked demonstrations. The transfer table (≈2 k episodes per world), the 4/4 jitter repeats and the 708-episode campaign carry the statistics, but the campaign fixes the torch seed at 42 and varies spawn yaw (4–10 repeats per map) and map geometry instead; a multi-seed campaign remains future work. The five presentation maps are hand-picked, and those runs are existence demonstrations with frame-exact audits, not distributions. Run-to-run nondeterminism is disclosed wherever it changed behaviour (the two tour paths through the goal-3 pocket).
- A competence seam the interface cannot close. Narrow-door and offset-gap geometry still defeats the frozen policy: serial rooms 0/3, campaign corridors 0.000 and gates 0.31, held-out map 0/10. The shield converts those failures from “collide” into “stop short” or “time out” (83 of the trained arm's 84 campaign failures are timeouts), which is an acceptable failure mode but still a failure.
- Mechanism-level negatives. The quadratic Koopman lift barely beats an affine fit at this scale (source v_x R^2 0.79 versus 0.77) and is never closed around the plant; DOB-lite is a scalar plant-gain observer, not a full disturbance observer, and it assumes the degradation is a gain; the single-cone barrier is blind to side contacts; park-after-success is interface policy rather than learned behaviour, and the frozen policy would grind a goal pocket indefinitely without it.
- Scope of embodiment. One robot class (differential drive, planar LiDAR, velocity-level interface), no manipulation, and the cross-embodiment result is deploy-time transfer through a message boundary rather than cross-embodiment learning.

C. How this serves the “learning can be transferred” goal

Transfer learning already shows that learning transfers when target data is available; this work is an answer for the regime where it is not, or where spending it is unsafe or expensive. Its service to the broader goal has four parts. It *prices* transfer: the adaptation-budget crossing states what frozen transfer saves in target-world demonstrations (≈238 k, with the single-seed and one-map caveats printed beside it). It *localises* transfer: the ablations and the ladder say which interface carries which axis, and place the boundary in behavioural rather than visual terms, since the mid rung is solved faster than the easy rung because it overlaps the source behaviour set more, not less. It *certifies* transfer: the guarantee arm converts “the policy usually behaves” into “no command reaches the chassis unless a forward simulation of the full hull cleared it”, at a measured 3.6 ms on CPU, with the residual risk enumerated rather than footnoted. And it *bounds* transfer: the held-out map measures exactly where learning stops paying and mathematical structure takes over, with competence 0/10, guarantee 0/10 contacts, same checkpoint, same shield. None of this replaces transfer learning. It delineates the part of the cross-world burden that mathematical structure can carry while the weights stand still, and it says which part it cannot.

VIII. CONCLUSION AND FUTURE WORK

We presented a benchmark study of the transfer-first paradigm: one frozen differential-drive policy, trained once, redeployed across unseen geometry, task structure, degraded dynamics and a second embodiment through adaptable mathematical interfaces, and wrapped at deployment in a three-tier safety shield whose no-contact property is structural, cost-measured (0.31 ms pass-through, 3.6 ms for a full 56-candidate batch on CPU against a 50 ms budget at 20 Hz) and audited frame-by-frame against exact wall geometry.

The positives are a zero-shot gradient on user-drawn maps, 4/4 under spawn jitter, dynamics recovery 0.028 → 0.363 within 3.2 % of an oracle, zero contact frames on the five presentation maps, and at campaign scale a shield-wrapped planner at 0.979 success with 0/236 collisions where its unprotected twin collides in 53.4 %, carrying to a held-out user-drawn map at 10/10 success and 0/10 collisions (0/246 cumulative) where the frozen policy reaches 0/10. The negatives are the wedge seam, the no-op and never-firing filter variants, a lift that barely beats affine, a livelock and its fix, an absorbing freeze and its fix, a hysteresis fix that was reverted, the trained arm's corridor and gates competence gap, and its single uncertified tier-3 scrape. Both sets are reported with the same instruments, from the same kept artifacts. The division of labour is the finding: guarantees came from the interface and were arm-agnostic; competence came from whichever controller was wrapped, and transferred along some axes and not others.

Future work is queued and not claimed. Certify the true contact envelope: widen the certified hull to the wheel track and casters, closing the hull-model gap that separates our two contact instruments. Certify or gate tier 3: clearance-gated commitment (a tie-break-only bonus, or sign-commitment with a minimum raw clearance floor) is the untested middle path between the v7.5 dither and the reverted v7.6 lock-in. Make the evidence dynamic: bounded-staleness decay in the persisted raster, so that moved obstacles are forgotten on a schedule and the guarantee survives non-static worlds. Widen the statistics: a multi-seed campaign and a family of held-out drawings rather than one, to turn the 0/246 guarantee record into a distribution. Close the model loop: few-shot lift refits used in control rather than in one-step prediction, and a disturbance observer that estimates more than a scalar gain. Remove the map oracle: build the geodesic field and Stack A's distance field from a SLAM estimate with quantified error, rather than from the file that defines the world. Hardware: the stack is edge-shaped by construction, a plain ROS 2 `/cmd_vel` boundary and a 3.6 ms CPU shield, so the physical run is an integration problem rather than a redesign, but it is unrun and we do not claim it. And on the framework side: while the initial NMTE architecture proposed Gromov–Wasserstein alignment for cross-embodiment transfer, the empirical results here show that a velocity-level interface was sufficient for the tested domains, so GW remains queued for aligning complex manipulation latent spaces, where a velocity boundary is not enough.

REFERENCES

[1] N. Jaquier *et al.*, “Transfer learning in robotics: An upcoming breakthrough? A review of promises and challenges,” *Int. J. Robotics Research*, vol. 44, pp. 465–485, 2024. DOI: 10.48550/arXiv.2311.18044.

[2] A. Shakerimov, T. Alizadeh, and H. A. Varol, “Efficient sim-to-real transfer in reinforcement learning through domain randomization and domain adaptation,” *IEEE Access*, vol. 11, pp. 136814–136825, 2023.

[3] T. Wang, D. Bhatt, X. Wang, and N. Atanasov, “Cross-embodiment robot manipulation skill transfer using latent space alignment,” arXiv preprint arXiv:2406.01968, 2024.

[4] D. Kalaria and K. Karydis, “Disturbance observer-based control barrier functions with residual model learning for safe reinforcement learning,” arXiv preprint arXiv:2410.06570, 2024.

[5] Y.-H. Chen, S. Liu, W. Xiao, C. Belta, and M. Otte, “Control barrier functions via Minkowski operations for safe navigation among polytopic sets,” in *Proc. IEEE 64th Conf. Decision and Control (CDC)*, 2025. DOI: 10.1109/CDC57313.2025.11312188.

[6] L. Shi, Z. Liu, and K. Karydis, “Koopman operators for modeling and control of soft robotics,” *Current Robotics Reports*, vol. 4, pp. 23–31, 2023. DOI: 10.1007/s43154-023-00099-8.

[7] A. K. Li, T. C. Silva, V. Edwards, V. Kumar, and M. A. Hsieh, “KoopMotion: Learning almost divergence free Koopman flow fields for motion planning,” *Proc. Machine Learning Research (PMLR)*, vol. 305, 2025. DOI: 10.48550/arXiv.2509.09074.

[8] D. Fox, W. Burgard, and S. Thrun, “The dynamic window approach to collision avoidance,” *IEEE Robotics & Automation Magazine*, vol. 4, no. 1, pp. 23–33, 1997.

[9] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, “Control barrier functions: Theory and applications,” in *Proc. 18th European Control Conf. (ECC)*, 2019, pp. 3420–3431.

[10] S. Ross, G. J. Gordon, and D. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *Proc. 14th Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, 2011, pp. 627–635.

[11] M. Mittal, C. Yu, Q. Yu, J. Liu, N. Rudin, and M. Hutter, “Orbit: A unified and modular framework for robot learning environments,” in *Proc. 5th Conf. Robot Learning (CoRL)*, 2021.

[12] M. Mittal *et al.* (NVIDIA), “Isaac Lab: A GPU-accelerated simulation framework for multi-modal robot learning,” technical report, arXiv:2511.04831, 2025.

[13] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” arXiv preprint arXiv:1707.06347, 2017.

[14] S. L. Brunton, M. Budišić, E. Kaiser, and J. N. Kutz, “Modern Koopman theory for dynamical systems,” *SIAM Review*, vol. 64, no. 4, pp. 729–840, 2022.

[15] W.-H. Chen, D. J. Ballance, P. J. Gawthrop, and J. J. Gribble, “A nonlinear disturbance observer for robotic manipulators,” *IEEE Trans. Industrial Electronics*, vol. 47, no. 4, pp. 806–814, 2000.

[16] A. V. Pitkevich and I. Makarov, “A survey of sim-to-real transfer methods for robotic manipulation,” in *Proc. IEEE 22nd Int. Symp. Intelligent Systems and Informatics (SISY)*, 2024. DOI: 10.1109/SISY62279.2024.10737545.

[17] D. Y. Patrashko and V. Gurau, “Machine learning-powered vision for robotic inspection in manufacturing: A review,” *Sensors*, vol. 26, no. 3, p. 788, 2026. DOI: 10.3390/s26030788.

[18] Q. Li, F. Cicirelli, A. Vinci, A. Guerrieri, W. Qi, and G. Fortino, “Quadruped robots: Bridging mechanical design, control, and applications,” *Robotics*, vol. 14, no. 5, p. 57, 2025. DOI: 10.3390/robotics14050057.

[19] G. Pan *et al.*, “RoboDuet: A framework affording mobile-manipulation and cross-embodiment,” arXiv preprint arXiv:2403.17367, 2024.

[20] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot Operating System 2: Design, architecture, and uses in the wild,” *Science Robotics*, vol. 7, no. 66, eabm7407, 2022.